LandInvest ERP Suite

Complete Technical & Business Handover Documentation

Document Version: 1.0.0
Last Verified: 2026-11-12
Databases: control_plane (7 tables) + tenant_* (19 tables)
Scope: Architecture, DB Data Dictionaries, 14 Modules, 26 Reports, 79 Permissions, Setup & Operations

LandInvest — Complete Project Documentation

Version: 1.0.0 | **Last Verified:** 2026-11-12

Table of Contents

1. LandInvest Handover Documentation
2. 01. Executive Summary
3. 02. System Architecture
4. 03. Tech Stack
5. 04a. Database Overview
6. 04b. Control Plane Data Dictionary (`control_plane`)
7. 04c. Tenant Database Data Dictionary (`tenant_*`)
8. 04d. Entity Relationship Diagrams (ERD)
9. 04e. Indexes and Foreign Key Constraints
10. Module 01: Auth & Tenant Management
11. Module 02: Partners & Capital
12. Module 03: Cash & Bank Ledger (Treasury)
13. Module 04: Land & Acquisition
14. Module 05: Plots & Inventory
15. Module 06: Project Expenses
16. Module 07: Voucher Management
17. Module 08: Plot Sales & Dues
18. Module 09: Reports & Profit Distribution
19. Module 10: Audit Trail
20. Module 11: Users & Invitations
21. Module 12: Roles & Permissions
22. Module 13: Executive Dashboard & Fiscal Years (FY)
23. Module 14: SaaS Billing
24. 06. End-to-End Business Workflows
25. 07. Authentication, Roles & Permissions
26. 08. Business Rules Register
27. 09. API Routes & Server Actions Inventory
28. 10. Reports & Dashboard
29. 11. Background Jobs & Worker Architecture
30. 12. Configuration & Environment Variables
31. 13. Setup and Run Guide (Clean Machine)
32. 14. Deployment & Operations
33. 15. Security
34. 16. Testing
35. 17. Developer Guide
36. 18. Architecture & Design Decision Log
37. 19. Known Issues, Tech Debt & Roadmap

38. 20. Glossary (English & Bengali)

LandInvest Handover Documentation

Purpose: Master index and reading guide for the complete LandInvest ERP handover documentation.

Intended Reader: Non-technical Owner, Technical Lead, New Developer, Maintainer.

Document Version: 1.0.0

Last Verified: 2026-11-12

How to Read These Docs

- **For Business Owners & Non-Technical Readers:** Start with 01-executive-summary.md, 06-workflows.md, and 20-glossary.md.
- **For Technical Leads & Architects:** Read 02-architecture.md, 04a-overview.md, 15-security.md, and 18-decision-log.md.
- **For New Developers & Maintainers:** Start with 13-setup-and-run-guide.md, 17-developer-guide.md, 04c-tenant-tables.md, and 09-api-and-actions.md.

Complete Document Index

#	Document	Focus
00	00-README.md	Index, reading guide, verification notes
01	01-executive-summary.md	Plain-English product overview, status, risks
02	02-architecture.md	Multi-tenant design, request lifecycle, caching
03	03-tech-stack.md	Languages, frameworks, exact versions, `npm audit`
04a	04-database/04a-overview.md	DB totals, schema diff, money flow, conventions
04b	04-database/04b-control-plane-tables.md	`control_plane` data dictionary (7 tables)
04c	04-database/04c-tenant-tables.md	Tenant DB data dictionary (19 tables)
04d	04-database/04d-erd.md	Control plane, tenant overview, and module ERDs
04e	04-database/04e-indexes-and-constraints.md	Foreign keys, logical links, indexes
05	05-modules/	14 individual module specifications
06	06-workflows.md	14 end-to-end workflows & state machines
07	07-auth-roles-permissions.md	Auth, sessions, 79 permission keys, role matrix
08	08-business-rules.md	Numbered register of enforced business rules
09	09-api-and-actions.md	39 APIs, 11 server action files, 37 pages
10	10-reports-and-dashboard.md	26 reports, 6 dashboard tabs, widget registry
11	11-background-jobs.md	BullMQ worker, provisioning, scheduled jobs
12	12-configuration-and-environment.md	Environment variables, settings tables, flags
13	13-setup-and-run-guide.md	Clean setup steps & real troubleshooting

#	Document	Focus
14	14-deployment-and-operations.md	Production deployment, backups, multi-tenant migrations
15	15-security.md	Tenant isolation, RBAC, PII, audit trail, risks
16	16-testing.md	Acceptance, FY, full E2E, i18n, and coverage gaps
17	17-developer-guide.md	Patterns, conventions, step-by-step recipes
18	18-decision-log.md	Architectural & domain decisions with trade-offs
19	19-known-issues-and-roadmap.md	Stubs, tech debt, risks, prioritized next steps
20	20-glossary.md	English & Bengali business and technical terms
—	PROJECT_DOCUMENTATION.md	Single combined document for offline reading & PDF

Regenerating Factual Sections

Run the following command whenever the database schema, routes, dependencies, or registries change:

```
npm run docs:generate
```

This executes `docs/_generated/extract_facts.mjs` (introspecting `control_plane` and `tenant_demo` plus TypeScript registries) and `docs/_generated/render_factual_docs.mjs`.

`UNVERIFIED:` and `Inferred reason:` Register

For strict accuracy, all items not directly proven by code comments or commit messages are listed below:

1. `Inferred reason:` **Database-per-tenant architecture** (`02-architecture.md`, `18-decision-log.md`): Chosen to provide hard physical isolation between competing land syndicates and simplify per-tenant backup/restore, at the cost of multi-DB migration complexity.
2. `Inferred reason:` **Native Windows MySQL + Redis instead of Docker** (`13-setup-and-run-guide.md`, `18-decision-log.md`): Observed in `src/worker.ts` error messages (`net start MySQL84`, `Memurai/redis-windows 6.2+`) and `package.json` (`next dev --webpack`), indicating a native Windows developer environment without Docker Desktop.
3. `Inferred reason:` **Stubbed Billing module** (`01-executive-summary.md`, `19-known-issues-and-roadmap.md`): `src/app/api/billing/pay/route.ts` and `billing_invoices` exist to model SaaS subscriptions without live bKash/SSLCommerz/Stripe gateway keys yet.
4. `UNVERIFIED:` **Production Linux deployment & HTTPS reverse proxy**: No production `Dockerfile`, `nginx.conf`, or `ecosystem.config.js` exists in the repository; production setup in `14-deployment-and-operations.md` is marked `NOT IMPLEMENTED`.
5. `UNVERIFIED:` **Plot hold expiry background worker**: `src/worker.ts` currently runs only the `tenant-provisioning` BullMQ worker; automated plot hold expiry or recurring snapshot cron jobs were not found in `src/worker.ts`.
6. `UNVERIFIED:` **Automated backup/restore execution in production**: Backup/restore procedures in `14-deployment-and-operations.md` document standard `mysqldump` commands for the observed schema layout, as no automated backup script exists in `scripts/`.

`;

writeDoc("00-README.md", ...)`

---

01. Executive Summary

Purpose: Plain-language overview of LandInvest for business owners, investors, and stakeholders.

Intended Reader: Non-technical Owner, Managing Partner, Stakeholder.

Last Verified: 2026-11-12

What Is LandInvest?

LandInvest is a bilingual (English and Bengali) multi-company software platform built specifically for **land investment syndicates and real estate developers in Bangladesh**. It replaces scattered Excel spreadsheets and paper cashbooks with one audited financial system.

Who Is It For & What Problem Does It Solve?

In Bangladeshi land syndicates, multiple partners pool capital to buy raw land, pay registration and development costs, subdivide the land into plots (measured in *Shotok*, *Katha*, and *Bigha*), sell plots on advance + installment plans, and split the net profit by ownership percentage.

Without a unified system, syndicates struggle with:

- Knowing exact cash in hand vs. bank balances at any moment.
- Tracking who paid for an expense personally (*Paid by Partner*) versus out of company cash.
- Preventing double-booking or selling plots above available area.
- Calculating fair partner profit shares and defending numbers during an audit.

LandInvest solves these problems by enforcing every financial movement through immutable ledgers and an automatic audit trail.

Main Modules (At a Glance)

Module	What It Does
Executive Dashboard	Live KPIs, cash/bank summaries, dues, and customizable role-based charts.
Partners & Capital	Tracks partner profiles, capital deposits, withdrawals, direct expenses, and profit credits.
Cash & Bank Treasury	Manages company bank accounts, cash-in-hand, and Cash ↔ Bank contra transfers with approval.
Land & Acquisition	Records land projects, Bangladeshi land deed metadata (Mouza, JL, Dag, Khatian), and partner share splits (must total 100%).
Plot Inventory	Subdivides projects into plots, tracks availability (*Available*, *Booked*, *Sold*), and converts units.
Project Expenses & Vouchers	Logs land, legal, and development vouchers with receipt uploads and manager approval.
Plot Sales & Dues	Supports single-plot, multi-plot, and whole-project sales, installment schedules, and receipt printing.
Reports & Profit Distribution	26 built-in financial, partner, project, sales, profit, and audit reports with PDF/Excel export and Fiscal Year filtering.
Users, Roles & Audit Trail	Fine-grained access control (79 permissions) and a complete log of who changed what and when.

Module	What It Does
Fiscal Year (FY) Settings	Lets administrators define Fiscal Years (e.g., Jul–Jun) and switch the active reporting FY.
Billing (SaaS)	Manages subscription plans and invoices for each tenant company (currently stubbed for manual testing).

Customer Onboarding & Subscription Model

1. **Multi-Tenant SaaS Design:** Each land company (*tenant*) gets its own isolated MySQL database so financial records never mix between customers.
2. **Onboarding:** New organizations can be provisioned via signup + background worker (`tenant-provisioning` queue in `src/worker.ts`), which creates the tenant database, pushes the schema, seeds default expense categories, creates 4 standard roles (*Owner*, *Managing Partner*, *Accountant*, *Site Manager*), and links the Owner account.
3. **Subscriptions:** `control_plane.subscription_plans` and `control_plane.billing_invoices` model monthly/annual SaaS plans per tenant (`trial` , `active` , `suspended` , `cancelled`).

Current Completion Status

Status	Areas
Done & Verified	Core ERP accounting (Partners, Treasury, Projects, Plots, Expenses, Vouchers, Multi-plot Sales, Collections, Profit Distribution), 79-permission RBAC, Bilingual UI (EN/BN), 26 Reports + PDF/Excel exports, Fiscal Year management, Full-project E2E & 92-check Acceptance suites.
Partly Done / Stubbed	SaaS Billing (`/api/billing/pay` is a stub without live bKash/SSLCommerz/Stripe integration); Email delivery for invites/password resets (uses token links/console in dev).
Not Yet Implemented	Automated production backup/restore scripts, Linux reverse-proxy/PM2 deployment manifests, automated plot-hold expiry background job.

Top Risks for an Owner to Know

1. **Billing & Email Are Not Live-Integrated:** Before onboarding paying public customers, wire a real payment gateway and transactional email provider (SMTP/SES/Resend).
2. **Database-Per-Tenant Operations:** Schema updates must be rolled out across every active tenant database, and backups must cover both `control_plane` and all `tenant_*` databases.
3. **Production Hardening Needed:** Ensure strong `NEXTAUTH_SECRET` , non-empty MySQL root/app passwords, and HTTPS termination before public internet exposure.

`;

---

02. System Architecture

Purpose: Explains LandInvest's multi-tenant architecture, request lifecycle, database client caching, and codebase layering.

Intended Reader: Technical Lead, Backend Developer, Architect.

Last Verified: 2026-11-12

High-Level System Architecture

MERMAID DIAGRAM

```
flowchart LR
  Browser["Browser (React 19 / Next.js UI)"] -->|HTTPS / HTTP| NextApp["Next.js 16 App Router\n(Pages, API Rou
  NextApp -->|Auth, Tenant Lookup, Billing| ControlDB[("MySQL: control_plane\n(7 tables)")]
  NextApp -->|Cached PrismaClient per tenantId| TenantDB[("MySQL: tenant_<slug>\n(19 tables per tenant)")]
  NextApp -->|Rate Limit & Enqueue Jobs| Redis[("Redis 6.2+\n(BullMQ Streams)")]
  Redis -->|tenant-provisioning| Worker["BullMQ Worker\n(src/worker.ts)"]
  Worker -->|CREATE DATABASE + prisma db push + seed| TenantDB
  Worker -->|Update tenant status| ControlDB
  NextApp -->|Voucher Uploads| Storage["Local File Storage\n(src/lib/storage.ts)"]
  NextApp -->|HTML -> PDF| Puppeteer["Puppeteer / Chromium\n(src/lib/reports/pdf.ts)"]
```

Multi-Tenant Design: Database-Per-Tenant

LandInvest uses a **hybrid control-plane + database-per-tenant** model:

- `control_plane` **(1 shared database):** Stores registered tenants (`tenants`), user login credentials (`global_users`), invitations, password resets, subscription plans, and billing invoices.
- * *`tenant_`* (1 database per customer):** Stores all operational ERP data (partners, projects, plots, expenses, sales, ledgers, roles, audit logs).

Why This Was Chosen (`Inferred reason:`)

- **Hard Data Isolation:** A bug in a SQL `WHERE` clause cannot leak one syndicate's financial data to another customer because their data lives in separate MySQL schemas.
- **Per-Customer Backup & Export:** An individual tenant database can be backed up, restored, or offboarded without touching other tenants.

Trade-Offs

- **Migrations Across Tenants:** Any change to `prisma/tenant/schema.prisma` must be pushed to *every* active tenant database.
- **Connection Management:** Each active tenant requires its own `PrismaClient` connection pool.

Request Lifecycle (Sequence)

MERMAID DIAGRAM

```
sequenceDiagram
  autonumber
  actor U as User (Browser)
  participant N as Next.js Route / RSC
  participant A as Auth.js (src/auth.ts)
  participant C as MySQL (control_plane)
  participant T as Tenant Client Cache (src/lib/tenant.ts)
  participant D as MySQL (tenant_db)

  U->>N: HTTP Request with JWT Cookie
  N->>A: getServerSession(authOptions)
  A->>C: Verify global_users.isActive & tenant.status
  alt Invalid / Suspended / Inactive
    C-->>A: Inactive or not found
    A-->>N: null session
    N-->>U: 401 Unauthorized / Redirect /login
  else Active User & Tenant
    C-->>A: Valid user + tenantId
    A-->>N: Session object
    N->>T: getTenantClientFromSession()
    T->>C: Lookup tenant dbName by tenantId
    T->>T: Return cached PrismaClient from globalThis.tenantClients Map
    N->>D: can(session, "permission.key") -> check user_roles & role_permissions
    alt Missing Permission
      N-->>U: 403 Forbidden / Forbidden UI Marker
    else Permitted
      N->>D: Execute business query / $transaction + writeAudit()
      D-->>N: Result
      N-->>U: 200 JSON / Rendered HTML
    end
  end
```

Tenant Database Client Caching & Connection Limits

Implemented in `src/lib/tenant.ts`:

- `globalThis.tenantClients` holds a `Map<string, PrismaClient>` keyed by `tenant.id`.
- `getTenantClient(tenant)` checks `globalForTenant.tenantClients.get(tenant.id)` (and verifies `(cached as any).dashboardLayout` exists to handle hot-reload schema updates in dev) before instantiating a new `PrismaClient` with `mysql://${tenant.dbUser}:@localhost:3306/${tenant.dbName}`.
- **Note on Connection Limits:** No explicit `connection_limit` query parameter or LRU eviction is currently set on `globalThis.tenantClients` (noted in `19-known-issues-and-roadmap.md`).

Folder & Layer Overview

Layer	Location	Responsibility
App Router UI & Pages	`src/app/**/page.tsx`	Server Components (auth/RBAC checks + initial data fetch) and `*Client.tsx` interactive views.
API Routes	`src/app/api/**/route.ts`	REST/JSON & file-upload/PDF/Excel endpoints.
Core Domain & Rules	`src/lib/`	`tenantRules.ts`, `collections.ts`, `treasury.ts`, `fiscalYear.ts`, `rbac.ts`, `permissions.ts`, `format.ts`, `area.ts`.

Layer	Location	Responsibility
Reports & Dashboard	`src/lib/reports/`, `src/lib/dashboard/`	26 report query modules + PDF/Excel builders + widget registry.
Background Worker	`src/worker.ts`	BullMQ worker for `tenant-provisioning`.
Database Schemas	`prisma/control/schema.prisma`, `prisma/tenant/schema.prisma`	Separate Prisma schemas & generated clients (`src/generated/control`, `src/generated/tenant`).
Scripts & Tests	`scripts/`	Setup, seeding, acceptance (`acceptance.mjs`), Fiscal Year TDD (`test_fiscal_years.mjs`), Full E2E (`test_full_project.mjs`).

`;

---

03. Tech Stack

Purpose: Complete inventory of languages, frameworks, libraries, runtime versions, and known dependency vulnerabilities.

Intended Reader: Technical Lead, New Developer, DevOps / Maintainer.

Last Verified: 2026-11-12

Runtime & Platform Environment

Component	Observed Value	Notes
Language	TypeScript `^5` + SQL (MySQL)	Strict type checking via `npm run typecheck`
Runtime	Node.js `v24.19.0`	Native ESM (`"type": "module"` not set; `.mjs` for ESM scripts, `tsx` for TS scripts)
Package Manager	npm `11.17.0`	Lockfile: `package-lock.json`
Database Engine	MySQL 8.4 (InnoDB, `utf8mb4_unicode_ci`)	Native Windows service on `localhost:3306` (no Docker required)
Queue / Cache	Redis 6.2+ (`redis-windows`)	Required by BullMQ streams (`XADD`) on `localhost:6379`
OS Assumptions	Windows 10/11 (dev) / Linux (prod)	`next dev --webpack` used on Windows for stability

---

Dependencies (Extracted from `package.json` & `package-lock.json`)

Package	Locked Version	Declared Range	Kind	Purpose
`@prisma/client`	`6.19.3`	`^6.19.3`	dependency	Type-safe ORM with dual-schema support (control plane + per-tenant DBs)
`bcryptjs`	`3.0.3`	`^3.0.3`	dependency	Password hashing for Credentials authentication
`bullmq`	`6.3.8`	`^6.3.8`	dependency	Redis-backed job queue for asynchronous tenant provisioning
`ioredis`	`6.0.0`	`^6.0.0`	dependency	Redis client for BullMQ and rate limiting
`lucide-react`	`1.48.0`	`^1.48.0`	dependency	Consistent SVG icon set across the UI
`mysql2`	`3.24.4`	`^3.24.4`	dependency	Direct MySQL driver for provisioning, seeding, and maintenance scripts
`next`	`16.3.6`	`16.3.6`	dependency	Full-stack React framework (App Router, Server Components, API routes)
`next-auth`	`4.24.15`	`^4.24.15`	dependency	Authentication and JWT session management (Auth.js v4)
`puppeteer`	`25.12.0`	`^25.12.0`	dependency	Headless Chromium for server-side PDF report & documentation generation

Package	Locked Version	Declared Range	Kind	Purpose
`react`	`19.2.8`	`19.2.8`	dependency	UI component library
`react-dom`	`19.2.8`	`19.2.8`	dependency	React DOM renderer
`recharts`	`3.10.1`	`^3.10.1`	dependency	Interactive charts for executive and financial dashboards
`@tailwindcss/postcss`	`4.3.3`	`^4`	devDependency	Tailwind CSS v4 PostCSS integration
`@types/node`	`20.19.43`	`^20`	devDependency	TypeScript definitions for Node.js APIs
`@types/react`	`19.3.0`	`^19`	devDependency	TypeScript definitions for React
`@types/react-dom`	`19.3.0`	`^19`	devDependency	TypeScript definitions for React DOM
`eslint`	`9.39.5`	`^9`	devDependency	Static code linting
`eslint-config-next`	`16.3.6`	`16.3.6`	devDependency	Next.js recommended ESLint rules
`prisma`	`6.19.3`	`^6.19.3`	devDependency	Prisma CLI for schema generation and db push
`tailwindcss`	`4.3.3`	`^4`	devDependency	Utility-first CSS styling
`tsx`	`4.23.15`	`^4.23.15`	devDependency	TypeScript script runner for workers, seeding, and doc generation
`typescript`	`5.9.3`	`^5`	devDependency	Static type checking across app and scripts

---

Known Vulnerabilities (`npm audit`)

Total dependencies scanned: **613** (165 prod, 403 dev).

Summary: **17 vulnerabilities** (Critical: 1, High: 9, Moderate: 7, Low: 0).

Package	Severity	Advisory Summary	Fix Available
`@prisma/config`	high	deepmerge-ts	Yes
`@typescript-eslint/eslint-plugin`	moderate	@typescript-eslint/type-utils; @typescript-eslint/utils	Yes
`@typescript-eslint/parser`	moderate	@typescript-eslint/typescript-estree	Yes
`@typescript-eslint/type-utils`	moderate	@typescript-eslint/typescript-estree; @typescript-eslint/utils	Yes

Package	Severity	Advisory Summary	Fix Available
`@typescript-eslint/typescript-estree`	moderate	minimatch	Yes
`@typescript-eslint/utils`	moderate	@typescript-eslint/typescript-estree	Yes
`chromium-bidi`	high	zod	Yes
`deepmerge-ts`	high	DeepmergeTS has stack exhaustion when merging recursive object graphs	Yes
`minimatch`	moderate	minimatch has potential denial of service via globparts amplification	Yes
`next`	critical	Next.js: Pending `use cache` fill can leak Draft Mode content into regular responses and persisted pages; Next.js has cache poisoning of SSG and ISR pages in self-hosted applications	Yes
`prisma`	high	@prisma/config	Yes
`puppeteer`	high	chromium-bidi; puppeteer-core	Yes
`puppeteer-core`	high	chromium-bidi	Yes
`sharp`	high	sharp : Vulnerability in librsvg dependency CVE-2026-96889	Yes
`source-map-js`	high	source-map-js allows event-loop denial of service through indexed source-map section offsets	Yes
`typescript-eslint`	moderate	@typescript-eslint/eslint-plugin; @typescript-eslint/parser	Yes
`zod`	high	Zod can exhaust memory while validating oversized constrained collections	Yes

---

04a. Database Overview

Purpose: High-level summary of LandInvest's dual-database MySQL architecture, totals, conventions, and Prisma schema verification.

Intended Reader: Technical Lead, Database Administrator, New Developer.

Last Verified: 2026-11-12

Database Totals (Introspected Live from `information_schema`)

Database	Tables	Views	Foreign Keys	Indexes	Engine	Charset / Collation
`control_plane`	7	0	6	18	InnoDB	`utf8mb4` / `utf8mb4_unicode_ci`
`tenant_demo` (per-tenant)	19	0	17	57	InnoDB	`utf8mb4` / `utf8mb4_unicode_ci`
Combined Total	**26**	**0**	**23**	**75**	InnoDB	`utf8mb4`

Prisma Schema vs Live MySQL Verification

Schema	In Live MySQL but NOT in Prisma	In Prisma but NOT in Live MySQL
`prisma/control/schema.prisma`	None (100% match)	None (100% match)
`prisma/tenant/schema.prisma`	`default_categories`	None (100% match)

> **Schema Mismatch Note:** `default_categories` is created directly via raw SQL in `src/worker.ts` (`seedDefaultCategories`) during tenant provisioning rather than via `prisma/tenant/schema.prisma`.

Naming & Type Conventions

- **ID Strategy:** UUID v4 strings stored as `CHAR(36)` generated in application code via `crypto.randomUUID()`.
- **Money Type:** `DECIMAL(15, 2)` (or `DECIMAL(18, 2)` for larger aggregates); formatted in BDT using lakh/crore grouping in `src/lib/format.ts`.
- **Land Area Unit:** Base unit is **Shotok** (`DECIMAL(10, 2)` / `DECIMAL(12, 4)`); converted to Katha (`1 Katha = 1.65 Shotok`) and Bigha (`1 Bigha = 33 Shotok`) in `src/lib/area.ts` (*verify exact factor in 08-business-rules.md*).
- **Timestamps:** `DATETIME(3)` / `TIMESTAMP` in UTC/server time.
- **Logical Cross-Database Links:** Because tenant databases are separate MySQL schemas, cross-DB links (e.g., `tenant.user_roles.user_id -> control_plane.global_users.id`, `tenant.partners.global_user_id`, `created_by`, `approved_by`, `changed_by`) are validated in application code rather than MySQL foreign key constraints.

Money Data Flow by Business Event

Business Event	Tables Written	Tables Read
Partner Capital Deposit	`partner_transactions`, `treasury_ledger`, `bank_accounts` (if BANK), `audit_log`	`partners`, `bank_accounts`

Business Event	Tables Written	Tables Read
Partner Withdrawal	`partner_transactions`, `treasury_ledger`, `bank_accounts` (if BANK), `audit_log`	`partners`, `treasury_ledger`
Expense Create	`project_expenses`, `audit_log`	`land_projects`, `bank_accounts`, `partners`
Expense Approve (CASH/BANK)	`project_expenses`, `treasury_ledger`, `bank_accounts` (if BANK), `audit_log`	`project_expenses`, `treasury_ledger`
Expense Approve (PAID_BY_PARTNER)	`project_expenses`, `partner_transactions` (`DIRECT_EXPENSE`), `audit_log`	`project_expenses`, `partners`
Plot Sale + Advance	`land_sales`, `project_plots`, `installment_schedule`, `customer_payments`, `treasury_ledger`, `bank_accounts`, `audit_log`	`project_plots`, `bank_accounts`
Installment Collection	`customer_payments`, `land_sales`, `installment_schedule` (FIFO), `project_plots` (to `SOLD` when due=0), `treasury_ledger`, `bank_accounts`, `audit_log`	`land_sales`, `installment_schedule`
Contra Transfer	`treasury_transfer_requests`, `treasury_ledger`, `bank_accounts`, `audit_log`	`treasury_ledger`, `bank_accounts`
Profit Distribution	`profit_distributions`, `partner_transactions` (`PROFIT_CREDIT`), `audit_log`	`land_projects`, `project_partners`, `land_sales`, `project_expenses`

04b. Control Plane Data Dictionary (`control_plane`)

Purpose: Column-level data dictionary for all 7 tables in `control_plane`.

Intended Reader: Backend Developer, DBA.

Last Verified: 2026-11-12

`billing_invoices`

- **Module:** Billing
- **Purpose:** Subscription invoices per tenant (currently stubbed manual/mock payment flow).
- **Primary Key:** `id` (char(36)) | **Engine:** InnoDB | **Collation:** utf8mb4_unicode_ci

Column	Type	Nullable	Default	Key	Notes
`id`	`char(36)`	NO	NULL	PK	-
`tenant_id`	`char(36)`	NO	NULL	FK -> `tenants.id`	-
`subscription_id`	`char(36)`	YES	NULL	FK -> `subscriptions.id`	-
`amount`	`decimal(10,2)`	NO	NULL	-	-
`currency`	`varchar(10)`	NO	`BDT`	-	-
`status`	`varchar(20)`	YES	NULL	-	-
`invoice_date`	`date`	NO	NULL	-	-
`provider_invoice_id`	`varchar(255)`	YES	NULL	-	-

`global_users`

- **Module:** Auth & Tenant Management
- **Purpose:** Global login identities mapped to a single tenant, storing password hash and owner/active flags.
- **Primary Key:** `id` (char(36)) | **Engine:** InnoDB | **Collation:** utf8mb4_unicode_ci

Column	Type	Nullable	Default	Key	Notes
`id`	`char(36)`	NO	NULL	PK	-
`tenant_id`	`char(36)`	NO	NULL	FK -> `tenants.id`	-
`full_name`	`varchar(150)`	NO	NULL	-	-
`email`	`varchar(255)`	NO	NULL	UNIQUE	-
`password_hash`	`text`	NO	NULL	-	-
`is_owner`	`tinyint(1)`	NO	`0`	-	-
`is_active`	`tinyint(1)`	NO	`1`	-	-
`must_change_password`	`tinyint(1)`	NO	`0`	-	-
`last_login_at`	`timestamp`	YES	NULL	-	-
`created_at`	`timestamp`	NO	`CURRENT_TIMESTAMP`	-	-

`password_reset_tokens`

- **Module:** Auth & Tenant Management
- **Purpose:** One-time hashed tokens for self-service password resets.
- **Primary Key:** `id` (char(36)) | **Engine:** InnoDB | **Collation:** utf8mb4_unicode_ci

Column	Type	Nullable	Default	Key	Notes
`id`	`char(36)`	NO	NULL	PK	-
`user_id`	`char(36)`	NO	NULL	-	-
`token`	`varchar(255)`	NO	NULL	UNIQUE	-
`expires_at`	`timestamp`	NO	NULL	-	-
`created_at`	`timestamp`	NO	`CURRENT_TIMESTAMP`	-	-

`plans`

- **Module:** General
- **Purpose:** Database table.
- **Primary Key:** `id` (char(36)) | **Engine:** InnoDB | **Collation:** utf8mb4_unicode_ci

Column	Type	Nullable	Default	Key	Notes
`id`	`char(36)`	NO	NULL	PK	-
`name`	`varchar(50)`	NO	NULL	-	-
`price_monthly`	`decimal(10,2)`	NO	NULL	-	-
`price_yearly`	`decimal(10,2)`	YES	NULL	-	-
`max_users`	`int`	NO	`5`	-	-
`max_projects`	`int`	NO	`10`	-	-
`features`	`json`	YES	NULL	-	-

`subscriptions`

- **Module:** General
- **Purpose:** Database table.
- **Primary Key:** `id` (char(36)) | **Engine:** InnoDB | **Collation:** utf8mb4_unicode_ci

Column	Type	Nullable	Default	Key	Notes
`id`	`char(36)`	NO	NULL	PK	-
`tenant_id`	`char(36)`	NO	NULL	FK -> `tenants.id`	-
`plan_id`	`char(36)`	NO	NULL	FK -> `plans.id`	-
`billing_cycle`	`varchar(20)`	NO	NULL	-	-
`current_period_start`	`date`	YES	NULL	-	-
`current_period_end`	`date`	YES	NULL	-	-

Column	Type	Nullable	Default	Key	Notes
payment_provider	varchar(30)	YES	NULL	-	-
provider_subscription_id	varchar(255)	YES	NULL	-	-
status	varchar(20)	NO	active	-	-

tenants

- **Module:** Auth & Tenant Management
- **Purpose:** Registered customer organizations (tenants), database connection metadata, and subscription status.
- **Primary Key:** id (char(36)) | **Engine:** InnoDB | **Collation:** utf8mb4_unicode_ci

Column	Type	Nullable	Default	Key	Notes
id	char(36)	NO	NULL	PK	-
company_name	varchar(200)	NO	NULL	-	-
db_host	varchar(255)	NO	NULL	-	-
db_name	varchar(100)	NO	NULL	UNIQUE	-
db_user	varchar(100)	NO	NULL	-	-
db_password_encrypted	text	NO	NULL	-	-
status	varchar(20)	NO	trial	-	-
created_at	timestamp	NO	CURRENT_TIMESTAMP	-	-

user_invitations

- **Module:** Users & Invitations
- **Purpose:** Pending email/token invitations for onboarding new users into a tenant.
- **Primary Key:** id (char(36)) | **Engine:** InnoDB | **Collation:** utf8mb4_unicode_ci

Column	Type	Nullable	Default	Key	Notes
id	char(36)	NO	NULL	PK	-
tenant_id	char(36)	NO	NULL	FK -> tenants.id	-
email	varchar(255)	NO	NULL	-	-
role_id	char(36)	NO	NULL	-	-
invite_token	varchar(255)	NO	NULL	UNIQUE	-
expires_at	timestamp	NO	NULL	-	-
accepted	tinyint(1)	NO	0	-	-

04c. Tenant Database Data Dictionary (`tenant_*`)

Purpose: Column-level data dictionary for all 19 tables in a provisioned tenant database (verified against `tenant_demo`).

Intended Reader: Backend Developer, DBA.

Last Verified: 2026-11-12

`audit_log`

- **Module:** Audit Trail
- **Purpose:** Tenant-level immutable audit trail recording table, record ID, action, actor, old/new JSON.
- **Primary Key:** `id` (char(36)) | **Engine:** InnoDB | **Collation:** utf8mb4_unicode_ci

Column	Type	Nullable	Default	Key	Notes
`id`	`char(36)`	NO	NULL	PK	-
`table_name`	`varchar(100)`	NO	NULL	-	-
`record_id`	`char(36)`	NO	NULL	-	-
`action`	`varchar(20)`	NO	NULL	-	-
`changed_by`	`char(36)`	NO	NULL	-	-
`old_values`	`json`	YES	NULL	-	-
`new_values`	`json`	YES	NULL	-	-
`created_at`	`timestamp`	NO	`CURRENT_TIMESTAMP`	-	-

`bank_accounts`

- **Module:** Cash & Bank Ledger
- **Purpose:** Company bank accounts with cached current_balance synced from treasury_ledger.
- **Primary Key:** `id` (char(36)) | **Engine:** InnoDB | **Collation:** utf8mb4_unicode_ci

Column	Type	Nullable	Default	Key	Notes
`id`	`char(36)`	NO	NULL	PK	-
`account_type`	`varchar(20)`	NO	`BANK`	-	-
`bank_name`	`varchar(100)`	NO	NULL	-	-
`account_name`	`varchar(150)`	YES	NULL	-	-
`account_number`	`varchar(50)`	NO	NULL	UNIQUE	-
`branch_name`	`varchar(100)`	YES	NULL	-	-
`routing_number`	`varchar(50)`	YES	NULL	-	-
`opening_balance`	`decimal(15,2)`	NO	`0.00`	-	-
`current_balance`	`decimal(15,2)`	NO	`0.00`	-	-

Column	Type	Nullable	Default	Key	Notes
is_active	tinyint(1)	NO	1	-	-
created_at	timestamp	NO	CURRENT_TIMESTAMP	-	-

customer_payments

- **Module:** Plot Sales & Dues
- **Purpose:** Customer receipts (advance and installments) linked to land_sales and bank/cash.
- **Primary Key:** id (char(36)) | **Engine:** InnoDB | **Collation:** utf8mb4_unicode_ci

Column	Type	Nullable	Default	Key	Notes
id	char(36)	NO	NULL	PK	-
sale_id	char(36)	NO	NULL	FK -> land_sales.id	-
payment_date	date	NO	NULL	-	-
amount_paid	decimal(15,2)	NO	NULL	-	-
payment_method	varchar(20)	NO	NULL	-	-
bank_account_id	char(36)	YES	NULL	FK -> bank_accounts.id	-
receiptNo	varchar(50)	NO	NULL	UNIQUE	-
remarks	varchar(255)	YES	NULL	-	-

dashboard_layouts

- **Module:** Dashboard
- **Purpose:** Saved per-role or per-user dashboard tab and widget layout JSON.
- **Primary Key:** id (char(36)) | **Engine:** InnoDB | **Collation:** utf8mb4_unicode_ci

Column	Type	Nullable	Default	Key	Notes
id	char(36)	NO	NULL	PK	-
scope	varchar(10)	NO	NULL	-	-
scope_id	char(36)	NO	NULL	-	-
layout	json	NO	NULL	-	-
updated_by	char(36)	YES	NULL	-	-
updated_at	timestamp	YES	CURRENT_TIMESTAMP	-	-

dashboard_settings

- **Module:** Configuration
- **Purpose:** Tenant-scoped JSON settings store (alert thresholds, fiscal_years, etc.).
- **Primary Key:** id (char(36)) | **Engine:** InnoDB | **Collation:** utf8mb4_unicode_ci

Column	Type	Nullable	Default	Key	Notes
`id`	`char(36)`	NO	NULL	PK	-
`setting_key`	`varchar(100)`	NO	NULL	UNIQUE	-
`setting_value`	`json`	NO	NULL	-	-
`updated_by`	`char(36)`	YES	NULL	-	-
`updated_at`	`timestamp`	YES	`CURRENT_TIMESTAMP`	-	-

`default_categories`

- **Module:** Project Expenses
- **Purpose:** Seeded default expense categories created by provisioning worker (exists in MySQL, not in Prisma schema).
- **Primary Key:** `id` (char(36)) | **Engine:** InnoDB | **Collation:** utf8mb4_unicode_ci

Column	Type	Nullable	Default	Key	Notes
`id`	`char(36)`	NO	NULL	PK	-
`name`	`varchar(100)`	NO	NULL	UNIQUE	-
`kind`	`varchar(20)`	NO	NULL	-	-

`installment_schedule`

- **Module:** Plot Sales & Dues
- **Purpose:** Scheduled installment due dates and amounts settled FIFO by customer payments.
- **Primary Key:** `id` (char(36)) | **Engine:** InnoDB | **Collation:** utf8mb4_unicode_ci

Column	Type	Nullable	Default	Key	Notes
`id`	`char(36)`	NO	NULL	PK	-
`sale_id`	`char(36)`	NO	NULL	FK -> `land_sales.id`	-
`due_date`	`date`	NO	NULL	-	-
`due_amount`	`decimal(15,2)`	NO	NULL	-	-
`status`	`varchar(20)`	NO	`UPCOMING`	-	-

`land_projects`

- **Module:** Land & Acquisition
- **Purpose:** Real estate / land development projects with mouza, khatian, area, and status.
- **Primary Key:** `id` (char(36)) | **Engine:** InnoDB | **Collation:** utf8mb4_unicode_ci

Column	Type	Nullable	Default	Key	Notes
`id`	`char(36)`	NO	NULL	PK	-
`project_name`	`varchar(200)`	NO	NULL	-	-
`mouza`	`varchar(100)`	NO	NULL	-	-

Column	Type	Nullable	Default	Key	Notes
jl_number	varchar(50)	YES	NULL	-	-
dag_numbers	text	NO	NULL	-	-
khatian_numbers	text	NO	NULL	-	-
total_area_shotok	decimal(10,2)	NO	NULL	-	-
purchase_price	decimal(15,2)	NO	NULL	-	-
bayna_amount	decimal(15,2)	NO	0.00	-	-
registration_cost	decimal(15,2)	NO	0.00	-	-
mutation_tax_cost	decimal(15,2)	NO	0.00	-	-
status	varchar(30)	NO	UNDER_BAYNA	-	-
created_at	timestamp	NO	CURRENT_TIMESTAMP	-	-

land_sales

- **Module:** Plot Sales & Dues
- **Purpose:** Buyer sale contracts (single plot, multi-plot, or whole-project) tracking agreed price and due amount.
- **Primary Key:** id (char(36)) | **Engine:** InnoDB | **Collation:** utf8mb4_unicode_ci

Column	Type	Nullable	Default	Key	Notes
id	char(36)	NO	NULL	PK	-
project_id	char(36)	NO	NULL	FK -> land_projects.id	-
plot_id	char(36)	YES	NULL	FK -> project_plots.id	-
buyer_name	varchar(150)	NO	NULL	-	-
buyer_phone	varchar(20)	NO	NULL	-	-
buyer_nid	varchar(50)	YES	NULL	-	-
sale_date	date	NO	NULL	-	-
total_agreed_price	decimal(15,2)	NO	NULL	-	-
advance_booking_amount	decimal(15,2)	NO	0.00	-	-
current_due_amount	decimal(15,2)	NO	NULL	-	-
status	varchar(30)	NO	BOOKED	-	-

partner_transactions

- **Module:** Partners & Capital
- **Purpose:** Ledger of partner capital deposits, withdrawals, direct expenses, and profit credits.
- **Primary Key:** id (char(36)) | **Engine:** InnoDB | **Collation:** utf8mb4_unicode_ci

Column	Type	Nullable	Default	Key	Notes
`id`	`char(36)`	NO	NULL	PK	-
`partner_id`	`char(36)`	NO	NULL	FK -> `partners.id`	-
`txn_type`	`varchar(30)`	NO	NULL	-	-
`amount`	`decimal(15,2)`	NO	NULL	-	-
`txn_date`	`date`	NO	NULL	-	-
`reference_id`	`char(36)`	YES	NULL	-	-
`notes`	`text`	YES	NULL	-	-
`created_at`	`timestamp`	NO	`CURRENT_TIMESTAMP`	-	-

`partners`

- **Module:** Partners & Capital
- **Purpose:** Syndicate investors/partners with opening balances, payout bank details, and active status.
- **Primary Key:** `id` (char(36)) | **Engine:** InnoDB | **Collation:** utf8mb4_unicode_ci

Column	Type	Nullable	Default	Key	Notes
`id`	`char(36)`	NO	NULL	PK	-
`global_user_id`	`char(36)`	YES	NULL	-	-
`name`	`varchar(150)`	NO	NULL	-	-
`phone`	`varchar(20)`	NO	NULL	UNIQUE	-
`nid_number`	`varchar(50)`	YES	NULL	UNIQUE	-
`bank_name`	`varchar(100)`	YES	NULL	-	-
`bank_account_name`	`varchar(150)`	YES	NULL	-	-
`bank_account_number`	`varchar(50)`	YES	NULL	-	-
`bank_branch`	`varchar(100)`	YES	NULL	-	-
`bank_routing_number`	`varchar(50)`	YES	NULL	-	-
`is_active`	`tinyint(1)`	NO	`1`	-	-
`opening_balance`	`decimal(15,2)`	NO	`0.00`	-	-
`created_at`	`timestamp`	NO	`CURRENT_TIMESTAMP`	-	-

`profit_distributions`

- **Module:** Reports & Profit
- **Purpose:** Recorded profit distribution payouts per project and partner.
- **Primary Key:** `id` (char(36)) | **Engine:** InnoDB | **Collation:** utf8mb4_unicode_ci

Column	Type	Nullable	Default	Key	Notes
`id`	`char(36)`	NO	NULL	PK	-

Column	Type	Nullable	Default	Key	Notes
project_id	char(36)	NO	NULL	FK -> land_projects.id	-
distribution_date	date	NO	NULL	-	-
realization_basis	varchar(20)	NO	NULL	-	-
total_project_net_profit	decimal(15,2)	NO	NULL	-	-
partner_id	char(36)	NO	NULL	FK -> partners.id	-
partner_share_percent	decimal(5,2)	NO	NULL	-	-
payout_amount	decimal(15,2)	NO	NULL	-	-

project_expenses

- **Module:** Project Expenses
- **Purpose:** Project acquisition, legal, development, and overhead expense vouchers (PENDING/APPROVED/REJECTED).
- **Primary Key:** id (char(36)) | **Engine:** InnoDB | **Collation:** utf8mb4_unicode_ci

Column	Type	Nullable	Default	Key	Notes
id	char(36)	NO	NULL	PK	-
project_id	char(36)	YES	NULL	FK -> land_projects.id	-
expense_category	varchar(100)	NO	NULL	-	-
amount	decimal(15,2)	NO	NULL	-	-
expense_date	date	NO	NULL	-	-
description	text	YES	NULL	-	-
payment_channel	varchar(20)	NO	NULL	-	-
bank_account_id	char(36)	YES	NULL	FK -> bank_accounts.id	-
paid_by_partner_id	char(36)	YES	NULL	FK -> partners.id	-
voucher_image_url	text	YES	NULL	-	-
status	varchar(20)	NO	PENDING	-	-
approved_by	char(36)	YES	NULL	-	-
created_by	char(36)	NO	NULL	-	-
created_at	timestamp	NO	CURRENT_TIMESTAMP	-	-

project_partners

- **Module:** Land & Acquisition
- **Purpose:** Shareholding split (%) of partners per land project (must sum to 100%).
- **Primary Key:** id (char(36)) | **Engine:** InnoDB | **Collation:** utf8mb4_unicode_ci

Column	Type	Nullable	Default	Key	Notes
`id`	`char(36)`	NO	NULL	PK	-
`project_id`	`char(36)`	NO	NULL	FK -> `land_projects.id`	-
`partner_id`	`char(36)`	NO	NULL	FK -> `partners.id`	-
`share_percentage`	`decimal(5,2)`	NO	NULL	-	-

`project_plots`

- **Module:** Plots
- **Purpose:** Individual subdivided plots within a project with Shotok size, asking price, and status.
- **Primary Key:** `id` (char(36)) | **Engine:** InnoDB | **Collation:** utf8mb4_unicode_ci

Column	Type	Nullable	Default	Key	Notes
`id`	`char(36)`	NO	NULL	PK	-
`project_id`	`char(36)`	NO	NULL	FK -> `land_projects.id`	-
`plot_number`	`varchar(50)`	NO	NULL	-	-
`plot_size_shotok`	`decimal(10,2)`	NO	NULL	-	-
`asking_price`	`decimal(15,2)`	NO	NULL	-	-
`status`	`varchar(20)`	NO	`AVAILABLE`	-	-

`role_permissions`

- **Module:** Roles & Permissions
- **Purpose:** Permission keys assigned to each role.
- **Primary Key:** `role_id` (char(36)), `permission_key` (varchar(100)) | **Engine:** InnoDB | **Collation:** utf8mb4_unicode_ci

Column	Type	Nullable	Default	Key	Notes
`role_id`	`char(36)`	NO	NULL	PK, FK -> `roles.id`	-
`permission_key`	`varchar(100)`	NO	NULL	PK	-

`roles`

- **Module:** Roles & Permissions
- **Purpose:** Tenant-scoped RBAC roles (Owner, Managing Partner, Accountant, Site Manager, custom roles).
- **Primary Key:** `id` (char(36)) | **Engine:** InnoDB | **Collation:** utf8mb4_unicode_ci

Column	Type	Nullable	Default	Key	Notes
`id`	`char(36)`	NO	NULL	PK	-
`name`	`varchar(100)`	NO	NULL	UNIQUE	-
`description`	`varchar(255)`	YES	NULL	-	-
`is_system`	`tinyint(1)`	NO	`0`	-	-

Column	Type	Nullable	Default	Key	Notes
`created_by`	`char(36)`	YES	NULL	-	-
`created_at`	`timestamp`	NO	`CURRENT_TIMESTAMP`	-	-
`updated_at`	`timestamp`	NO	NULL	-	-

`treasury_ledger`

- **Module:** Cash & Bank Ledger
- **Purpose:** Immutable cash and bank cashbook ledger (CREDIT/DEBIT) for all treasury movements.
- **Primary Key:** `id` (char(36)) | **Engine:** InnoDB | **Collation:** utf8mb4_unicode_ci

Column	Type	Nullable	Default	Key	Notes
`id`	`char(36)`	NO	NULL	PK	-
`txn_date`	`date`	NO	NULL	-	-
`account_type`	`varchar(20)`	NO	NULL	-	-
`bank_account_id`	`char(36)`	YES	NULL	FK -> `bank_accounts.id`	-
`flow_type`	`varchar(10)`	NO	NULL	-	-
`amount`	`decimal(15,2)`	NO	NULL	-	-
`purpose`	`varchar(255)`	NO	NULL	-	-
`reference_type`	`varchar(30)`	NO	NULL	-	-
`reference_id`	`char(36)`	YES	NULL	-	-
`created_by`	`char(36)`	NO	NULL	-	-
`created_at`	`timestamp`	NO	`CURRENT_TIMESTAMP`	-	-

`user_roles`

- **Module:** Users & Roles
- **Purpose:** Maps control-plane global_users.id to a tenant role.
- **Primary Key:** `user_id` (char(36)) | **Engine:** InnoDB | **Collation:** utf8mb4_unicode_ci

Column	Type	Nullable	Default	Key	Notes
`user_id`	`char(36)`	NO	NULL	PK	-
`role_id`	`char(36)`	NO	NULL	FK -> `roles.id`	-
`assigned_by`	`char(36)`	YES	NULL	-	-
`assigned_at`	`timestamp`	NO	`CURRENT_TIMESTAMP`	-	-

04d. Entity Relationship Diagrams (ERD)

Purpose: Visual Mermaid ERDs for `control_plane`, the full `tenant_*` database, and each functional module.

Intended Reader: Technical Lead, Database Administrator, Backend Developer.

Last Verified: 2026-11-12

---

1. Control Plane ERD (\`control_plane\`)

MERMAID DIAGRAM

```
erDiagram
  tenants ||--o{ global_users : "has users"
  tenants ||--o{ user_invitations : "has invites"
  tenants ||--o{ billing_invoices : "billed via"
  tenants ||--o{ audit_logs : "logs"
  subscription_plans ||--o{ tenants : "subscribed to"
  global_users ||--o{ password_reset_tokens : "resets password"

  tenants {
    char36 id PK
    varchar slug UK
    varchar db_name UK
    varchar status
  }
  global_users {
    char36 id PK
    char36 tenant_id FK
    varchar email UK
    tinyint is_owner
    tinyint is_active
  }
  subscription_plans {
    char36 id PK
    varchar code UK
    decimal price_bdt
  }
  billing_invoices {
    char36 id PK
    char36 tenant_id FK
    varchar status
  }
```

---

2. Full Tenant Database Overview ERD (\`tenant_*\`)

MERMAID DIAGRAM

```
erDiagram
  partners ||--o{ partner_transactions : "ledger"
  partners ||--o{ project_partners : "owns share %"
  partners ||--o{ project_expenses : "paid directly"
  partners ||--o{ profit_distributions : "receives"

  land_projects ||--o{ project_partners : "syndicate split"
  land_projects ||--o{ project_plots : "subdivided into"
  land_projects ||--o{ project_expenses : "incurs"
  land_projects ||--o{ land_sales : "sells"
  land_projects ||--o{ profit_distributions : "distributes"

  project_plots ||--o{ land_sales : "sold via"
  land_sales ||--o{ installment_schedule : "scheduled"
  land_sales ||--o{ customer_payments : "settled by"

  bank_accounts ||--o{ treasury_ledger : "tracks"
  bank_accounts ||--o{ project_expenses : "pays"
  bank_accounts ||--o{ customer_payments : "receives"
  bank_accounts ||--o{ treasury_transfer_requests : "contra"

  roles ||--o{ role_permissions : "grants"
  roles ||--o{ user_roles : "assigned to"
```

---

3. Module ERDs

Partners & Capital

MERMAID DIAGRAM

```
erDiagram
  partners ||--o{ partner_transactions : "DEPOSIT / WITHDRAWAL / DIRECT_EXPENSE / PROFIT_CREDIT"
  partners {
    char36 id PK
    varchar name
    decimal opening_balance
    char36 global_user_id "Logical link to control_plane.global_users"
  }
  partner_transactions {
    char36 id PK
    char36 partner_id FK
    varchar txn_type
    decimal amount
    date txn_date
  }
```

Treasury (Cash & Bank)

MERMAID DIAGRAM

```
erDiagram
  bank_accounts ||--o{ treasury_ledger : "bank entries"
  bank_accounts ||--o{ treasury_transfer_requests : "contra target/source"
  bank_accounts {
    char36 id PK
    varchar bank_name
    varchar account_number UK
    decimal current_balance
  }
  treasury_ledger {
    char36 id PK
    varchar source_type "CASH or BANK"
    char36 bank_account_id FK
    varchar txn_type "CREDIT or DEBIT"
    decimal amount
    varchar reference_type
    char36 reference_id
  }
  treasury_transfer_requests {
    char36 id PK
    varchar from_type
    varchar to_type
    decimal amount
    varchar status "PENDING / APPROVED / REJECTED"
  }
```

Plots, Sales & Collections

MERMAID DIAGRAM

```
erDiagram
  land_projects ||--o{ project_plots : "has"
  project_plots ||--o{ land_sales : "primary plot"
  land_sales ||--o{ installment_schedule : "installments"
  land_sales ||--o{ customer_payments : "payments"
  land_sales {
    char36 id PK
    char36 project_id FK
    char36 plot_id FK
    decimal total_agreed_price
    decimal current_due_amount
    varchar status "BOOKED / PARTIAL / PAID_IN_FULL"
  }
  project_plots {
    char36 id PK
    varchar plot_number
    decimal plot_size_shotok
    varchar status "AVAILABLE / BOOKED / SOLD"
  }
```

RBAC & Dashboard

MERMAID DIAGRAM

```
erDiagram
  roles ||--o{ role_permissions : "has"
  roles ||--o{ user_roles : "assigned"
  roles {
    char36 id PK
    varchar name UK
    tinyint is_system
  }
  role_permissions {
    char36 id PK
    char36 role_id FK
    varchar permission_key
  }
  user_roles {
    char36 id PK
    char36 user_id "Logical link to global_users.id"
    char36 role_id FK
  }
  dashboard_layouts {
    char36 id PK
    varchar target_type "ROLE or USER"
    char36 target_id
    json layout_json
  }
```

`;

writeDoc("04-database/04d-erd.md", ...)`

04e. Indexes and Foreign Key Constraints

Purpose: Complete reference of all physical MySQL foreign keys, unique constraints, indexes, and logical cross-database links.

Intended Reader: Database Administrator, Backend Developer.

Last Verified: 2026-11-12

Control Plane Foreign Keys (`control_plane`: 6)

Child Table	Child Column	Parent Table	Parent Column	ON DELETE	ON UPDATE	Constraint Name
`billing_invoices`	`subscription_id`	`subscriptions`	`id`	SET NULL	CASCADE	`billing_invoices_subscription_id_fkey`
`billing_invoices`	`tenant_id`	`tenants`	`id`	RESTRICT	CASCADE	`billing_invoices_tenant_id_fkey`
`global_users`	`tenant_id`	`tenants`	`id`	RESTRICT	CASCADE	`global_users_tenant_id_fkey`
`subscriptions`	`plan_id`	`plans`	`id`	RESTRICT	CASCADE	`subscriptions_plan_id_fkey`
`subscriptions`	`tenant_id`	`tenants`	`id`	RESTRICT	CASCADE	`subscriptions_tenant_id_fkey`
`user_invitations`	`tenant_id`	`tenants`	`id`	RESTRICT	CASCADE	`user_invitations_tenant_id_fkey`

Tenant Database Foreign Keys (`tenant_demo`: 17)

Child Table	Child Column	Parent Table	Parent Column	ON DELETE	ON UPDATE	Constraint Name
`customer_payments`	`bank_account_id`	`bank_accounts`	`id`	SET NULL	CASCADE	`customer_payments_bank_account_id_fkey`
`customer_payments`	`sale_id`	`land_sales`	`id`	RESTRICT	CASCADE	`customer_payments_sale_id_fkey`
`installment_schedule`	`sale_id`	`land_sales`	`id`	RESTRICT	CASCADE	`installment_schedule_sale_id_fkey`
`land_sales`	`plot_id`	`project_plots`	`id`	SET NULL	CASCADE	`land_sales_plot_id_fkey`
`land_sales`	`project_id`	`land_projects`	`id`	RESTRICT	CASCADE	`land_sales_project_id_fkey`
`partner_transactions`	`partner_id`	`partners`	`id`	RESTRICT	CASCADE	`partner_transactions_partner_id_fkey`

Child Table	Child Column	Parent Table	Parent Column	ON DELETE	ON UPDATE	Constraint Name
profit_distributions	partner_id	partners	id	RESTRICT	CASCADE	profit_distributions_partner_id_fkey
profit_distributions	project_id	land_projects	id	RESTRICT	CASCADE	profit_distributions_project_id_fkey
project_expenses	bank_account_id	bank_accounts	id	SET NULL	CASCADE	project_expenses_bank_account_id_fkey
project_expenses	paid_by_partner_id	partners	id	SET NULL	CASCADE	project_expenses_paid_by_partner_id_fkey
project_expenses	project_id	land_projects	id	SET NULL	CASCADE	project_expenses_project_id_fkey
project_partners	partner_id	partners	id	RESTRICT	CASCADE	project_partners_partner_id_fkey
project_partners	project_id	land_projects	id	RESTRICT	CASCADE	project_partners_project_id_fkey
project_plots	project_id	land_projects	id	RESTRICT	CASCADE	project_plots_project_id_fkey
role_permissions	role_id	roles	id	CASCADE	CASCADE	role_permissions_role_id_fkey
treasury_ledger	bank_account_id	bank_accounts	id	SET NULL	CASCADE	treasury_ledger_bank_account_id_fkey
user_roles	role_id	roles	id	RESTRICT	CASCADE	user_roles_role_id_fkey

---

Logical Cross-Database Links (Not Physical Foreign Keys)

Tenant Table & Column	Target Table & Column	Enforced By
user_roles.user_id	control_plane.global_users.id	Application RBAC & session resolution
partners.global_user_id	control_plane.global_users.id	Partner account linking logic
project_expenses.created_by / approved_by	control_plane.global_users.id	Session user ID from JWT
treasury_ledger.created_by	control_plane.global_users.id	Session user ID from JWT
treasury_transfer_requests.requested_by / approved_by	control_plane.global_users.id	Session user ID from JWT
audit_log.changed_by	control_plane.global_users.id	writeAudit() in src/lib/tenantRules.ts

Tenant Table & Column	Target Table & Column	Enforced By
`dashboard_settings.updated_by`	`control_plane.global_users.id`	API route session user ID

Control Plane Indexes (18 indexes)

Table	Index Name	Kind	Columns	Type
`billing_invoices`	`billing_invoices_subscription_id_idx`	INDEX	`subscription_id`	BTREE
`billing_invoices`	`billing_invoices_tenant_id_idx`	INDEX	`tenant_id`	BTREE
`billing_invoices`	`PRIMARY`	UNIQUE	`id`	BTREE
`global_users`	`global_users_email_key`	UNIQUE	`email`	BTREE
`global_users`	`global_users_tenant_id_idx`	INDEX	`tenant_id`	BTREE
`global_users`	`PRIMARY`	UNIQUE	`id`	BTREE
`password_reset_tokens`	`password_reset_tokens_token_key`	UNIQUE	`token`	BTREE
`password_reset_tokens`	`password_reset_tokens_user_id_idx`	INDEX	`user_id`	BTREE
`password_reset_tokens`	`PRIMARY`	UNIQUE	`id`	BTREE
`plans`	`PRIMARY`	UNIQUE	`id`	BTREE
`subscriptions`	`PRIMARY`	UNIQUE	`id`	BTREE
`subscriptions`	`subscriptions_plan_id_idx`	INDEX	`plan_id`	BTREE
`subscriptions`	`subscriptions_tenant_id_idx`	INDEX	`tenant_id`	BTREE
`tenants`	`PRIMARY`	UNIQUE	`id`	BTREE
`tenants`	`tenants_db_name_key`	UNIQUE	`db_name`	BTREE
`user_invitations`	`PRIMARY`	UNIQUE	`id`	BTREE
`user_invitations`	`user_invitations_invite_token_key`	UNIQUE	`invite_token`	BTREE
`user_invitations`	`user_invitations_tenant_id_idx`	INDEX	`tenant_id`	BTREE

Tenant Database Indexes (57 indexes)

Table	Index Name	Kind	Columns	Type
`audit_log`	`audit_log_changed_by_idx`	INDEX	`changed_by`	BTREE
`audit_log`	`audit_log_created_at_idx`	INDEX	`created_at`	BTREE
`audit_log`	`audit_log_table_name_created_at_idx`	INDEX	`table_name`, `created_at`	BTREE
`audit_log`	`PRIMARY`	UNIQUE	`id`	BTREE

Table	Index Name	Kind	Columns	Type
bank_accounts	bank_accounts_account_number_key	UNIQUE	account_number	BTREE
bank_accounts	PRIMARY	UNIQUE	id	BTREE
customer_payments	customer_payments_bank_account_id_fkey	INDEX	bank_account_id	BTREE
customer_payments	customer_payments_payment_date_idx	INDEX	payment_date	BTREE
customer_payments	customer_payments_receiptNo_key	UNIQUE	receiptNo	BTREE
customer_payments	customer_payments_sale_id_idx	INDEX	sale_id	BTREE
customer_payments	PRIMARY	UNIQUE	id	BTREE
dashboard_layouts	PRIMARY	UNIQUE	id	BTREE
dashboard_layouts	uq_scope_scope_id	UNIQUE	scope, scope_id	BTREE
dashboard_settings	PRIMARY	UNIQUE	id	BTREE
dashboard_settings	setting_key	UNIQUE	setting_key	BTREE
default_categories	PRIMARY	UNIQUE	id	BTREE
default_categories	uq_cat_name	UNIQUE	name	BTREE
installment_schedule	installment_schedule_due_date_idx	INDEX	due_date	BTREE
installment_schedule	installment_schedule_sale_id_idx	INDEX	sale_id	BTREE
installment_schedule	installment_schedule_status_due_date_idx	INDEX	status, due_date	BTREE
installment_schedule	PRIMARY	UNIQUE	id	BTREE
land_projects	PRIMARY	UNIQUE	id	BTREE
land_sales	land_sales_plot_id_idx	INDEX	plot_id	BTREE
land_sales	land_sales_project_id_idx	INDEX	project_id	BTREE

Table	Index Name	Kind	Columns	Type
land_sales	land_sales_sale_date_idx	INDEX	sale_date	BTREE
land_sales	land_sales_status_sale_date_idx	INDEX	status, sale_date	BTREE
land_sales	PRIMARY	UNIQUE	id	BTREE
partner_transactions	partner_transactions_partner_id_idx	INDEX	partner_id	BTREE
partner_transactions	partner_transactions_txn_date_idx	INDEX	txn_date	BTREE
partner_transactions	PRIMARY	UNIQUE	id	BTREE
partners	partners_nid_number_key	UNIQUE	nid_number	BTREE
partners	partners_phone_key	UNIQUE	phone	BTREE
partners	PRIMARY	UNIQUE	id	BTREE
profit_distributions	PRIMARY	UNIQUE	id	BTREE
profit_distributions	profit_distributions_partner_id_idx	INDEX	partner_id	BTREE
profit_distributions	profit_distributions_project_id_idx	INDEX	project_id	BTREE
project_expenses	PRIMARY	UNIQUE	id	BTREE
project_expenses	project_expenses_bank_account_id_idx	INDEX	bank_account_id	BTREE
project_expenses	project_expenses_expense_date_idx	INDEX	expense_date	BTREE
project_expenses	project_expenses_paid_by_partner_id_idx	INDEX	paid_by_partner_id	BTREE
project_expenses	project_expenses_project_id_idx	INDEX	project_id	BTREE
project_expenses	project_expenses_status_expense_date_idx	INDEX	status, expense_date	BTREE
project_partners	PRIMARY	UNIQUE	id	BTREE
project_partners	project_partners_partner_id_idx	INDEX	partner_id	BTREE
project_partners	project_partners_project_id_partner_id_key	UNIQUE	project_id, partner_id	BTREE

Table	Index Name	Kind	Columns	Type
project_plots	PRIMARY	UNIQUE	id	BTREE
project_plots	project_plots_project_id_idx	INDEX	project_id	BTREE
role_permissions	PRIMARY	UNIQUE	role_id, permission_key	BTREE
roles	PRIMARY	UNIQUE	id	BTREE
roles	roles_name_key	UNIQUE	name	BTREE
treasury_ledger	PRIMARY	UNIQUE	id	BTREE
treasury_ledger	treasury_ledger_account_type_txn_date_idx	INDEX	account_type, txn_date	BTREE
treasury_ledger	treasury_ledger_bank_account_id_idx	INDEX	bank_account_id	BTREE
treasury_ledger	treasury_ledger_reference_type_reference_id_idx	INDEX	reference_type, reference_id	BTREE
treasury_ledger	treasury_ledger_txn_date_idx	INDEX	txn_date	BTREE
user_roles	PRIMARY	UNIQUE	user_id	BTREE
user_roles	user_roles_role_id_fkey	INDEX	role_id	BTREE

Module 01: Auth & Tenant Management

Last Verified: 2026-11-12

1. **Purpose:** Authenticates users, manages password resets and forced password changes, and provisions isolated tenant databases on signup.
2. **Who Uses It:** All users (`/login` , `/change-password` , `/forgot-password` , `/reset-password/[token]`); prospective owners (`/signup`).
3. **Screens & Routes:** `/login` , `/signup` , `/forgot-password` , `/reset-password/[token]` , `/change-password` , `/invite/[token]` .
4. **Main Workflow:**

MERMAID DIAGRAM

```
flowchart LR
  Login["/login"] --> Auth["src/auth.ts (Credentials)"]
  Auth --> CDB["control_plane.global_users + tenants"]
  CDB -->|Valid| JWT["Signed JWT Session"]
```

5. **Data (Tables Read & Written):** Reads/Writes `control_plane.tenants` , `control_plane.global_users` , `control_plane.password_reset_tokens` , `control_plane.user_invitations` .
6. **Business Rules:** Active check on every session callback (`src/auth.ts`); suspended/cancelled tenants blocked; rate limiting (`AUTH_RATE_LIMIT_MAX`).
7. **Calculations:** N/A (bcrypt cost factor hashing & token expiry timestamps).
8. **Integrations:** Provides `tenantId` , `user_id` , and `isOwner` to `src/lib/tenant.ts` and `src/lib/rbac.ts` .
9. **Code Map:** `src/auth.ts` , `src/app/login/page.tsx` , `src/app/signup/page.tsx` , `src/app/api/signup/route.ts` , `src/app/api/auth/*` , `src/worker.ts` .
10. **Edge Cases & Validations:** Deactivated users or suspended tenants are rejected on the next session check even if their JWT has not expired.
11. **Known Limitations:** Email delivery for password resets/invites is not wired to an external SMTP provider by default.

Module 02: Partners & Capital

Last Verified: 2026-11-12

1. **Purpose:** Tracks syndicate investors, opening equity, capital deposits, withdrawals, direct project expenses, and profit credits.
2. **Who Uses It:** Owner, Managing Partner (`partners.view`, `partners.manage`, `partners.capital.deposit`, `partners.capital.withdraw`, `partners.ledger.view`).
3. **Screens & Routes:** `/partners` (directory), `/partners/dashboard`, `/partners/capital` (deposits & withdrawals).
4. **Main Workflow:**

MERMAID DIAGRAM

```
flowchart LR
  Cap["/partners/capital"] --> TX["$transaction"]
  TX --> PT["partner_transactions"]
  TX --> TL["treasury_ledger"]
```

5. **Data (Tables Read & Written):** Reads/Writes `partners`, `partner_transactions`, `treasury_ledger`, `bank_accounts`, `audit_log`.
6. **Business Rules:** Partner capital deposits/withdrawals must update `treasury_ledger` (and `bank_accounts` if BANK) atomically with `audit_log` (`BR-10`, `BR-11`).
7. **Calculations: Worked Example:** Opening `৳ 50,00,000` + Deposit `৳ 10,00,000` + Profit Credit `৳ 4,50,000` - Direct Expense `৳ 50,000` - Withdrawal `৳ 2,00,000` = Net Capital `৳ 62,00,000`.
8. **Integrations:** Feeds `project_partners` share splits, `project_expenses` (`PAID_BY_PARTNER`), `profit_distributions`, and Partner reports.
9. **Code Map:** `src/app/partners/*`, *`src/app/api/partners/route.ts`*, *`src/lib/reports/partner-`*`.ts`.
10. **Edge Cases & Validations:** Opening balance must be included alongside `partner_transactions` when computing total partner net position.
11. **Known Limitations:** `POST` mutations use Server Actions rather than `/api/partners` (which is GET-only).

Module 03: Cash & Bank Ledger (Treasury)

Last Verified: 2026-11-12

1. **Purpose:** Maintains real-time Cash-in-Hand and Bank account balances through an immutable CREDIT/DEBIT ledger and approved contra transfers.
2. **Who Uses It:** Owner, Managing Partner, Accountant (`treasury.view` , `treasury.bank.manage` , `treasury.contra.create` , `treasury.contra.approve`).
3. **Screens & Routes:** `/treasury` , `/treasury/dashboard` , `/treasury/accounts` , `/treasury/transactions` , `/treasury/approvals` .
4. **Main Workflow:**

MERMAID DIAGRAM

```
flowchart LR
  Acct["Accountant Contra"] -->|Pending| Req["treasury_transfer_requests"]
  Req -->|Owner Approves| Led["treasury_ledger + bank_accounts"]
```

5. **Data (Tables Read & Written):** Reads/Writes `bank_accounts` , `treasury_ledger` , `treasury_transfer_requests` , `audit_log` .
6. **Business Rules:** Non-approvers queue contra transfers as `PENDING` ; approvers post paired `DEBIT` and `CREDIT` rows (`BR-13`). `PAID_BY_PARTNER` never writes `treasury_ledger` (`BR-01`).
7. **Calculations: Worked Example:** Cash Opening `৳ 5,00,000` + Sale Advance `৳ 3,00,000` - Contra to Bank `৳ 2,00,000` = Cash Balance `৳ 6,00,000` ; Bank increases by `+৳ 2,00,000` .
8. **Integrations:** Receives debits from approved `CASH` / `BANK` expenses and credits from `customer_payments` and partner deposits.
9. **Code Map:** `src/lib/treasury.ts` , `src/app/treasury/` , **`src/app/api/banks/route.ts`** , **`src/app/api/treasury/contra/`** .
10. **Edge Cases & Validations:** Contra transfers requiring a bank account validate `bankAccountId` exists and is active.
11. **Known Limitations:** Bank reconciliation against external bank CSV statements is not yet automated.

Module 04: Land & Acquisition

Last Verified: 2026-11-12

1. **Purpose:** Manages land development projects, Bangladeshi deed metadata (Mouza, JL, Dag, Khatian), and partner ownership percentages.
2. **Who Uses It:** Owner, Managing Partner, Site Manager (`projects.view` , `projects.create` , `projects.edit` , `projects.delete`).
3. **Screens & Routes:** `/projects` , `/projects/new` (acquisition wizard), `/projects/[id]` .
4. **Main Workflow:**

MERMAID DIAGRAM

```
flowchart LR
  Wiz["/projects/new"] --> LP["land_projects"]
  Wiz --> PP["project_partners (100%)"]
  Wiz --> Plots["project_plots (optional)"]
```

5. **Data (Tables Read & Written):** Reads/Writes `land_projects` , `project_partners` , `project_plots` , `audit_log` .
6. **Business Rules:** Partner `share_percent` per project must sum to 100% (`BR-03`). Area stored in Shotok (`BR-12`).
7. **Calculations: Worked Example:** Acquired `66 Shotok` (`2 Bigha` / `40 Katha`). Partner A (`60%`) + Partner B (`40%`) = `100%` .
8. **Integrations:** Parent entity for `project_plots` , `project_expenses` , `land_sales` , and `profit_distributions` .
9. **Code Map:** `src/app/projects/**` , `src/app/api/projects/route.ts` , `src/lib/area.ts` .
10. **Edge Cases & Validations:** Land acquisition costs (Bayna, Registration) are recorded as `project_expenses` linked to `project_id` .
11. **Known Limitations:** Project-level document file attachments (scanned deeds) are limited compared to expense vouchers.

Module 05: Plots & Inventory

Last Verified: 2026-11-12

1. **Purpose:** Subdivides acquired project land into saleable plots, tracks Shotok size and asking price, and locks sold/booked inventory.
2. **Who Uses It:** Owner, Managing Partner, Site Manager (`plots.manage` , `projects.view`).
3. **Screens & Routes:** `/projects/plots` , `/projects/[id]` .
4. **Main Workflow:**

MERMAID DIAGRAM

```
flowchart LR
  Avail["AVAILABLE"] -->|Installment Sale| Booked["BOOKED"]
  Avail -->|100% Upfront Sale| Sold["SOLD"]
  Booked -->|Due = 0| Sold
```

5. **Data (Tables Read & Written):** Reads/Writes `project_plots` , `land_projects` , `audit_log` .
6. **Business Rules:** Total subdivided plot Shotok cannot exceed project saleable area (`BR-04`); only `AVAILABLE` plots can be booked/sold (`BR-05`).
7. **Calculations: Worked Example:** `5.00 Shotok` plot @ `৳ 2,50,000/Shotok` = Asking Price `৳ 12,50,000.00` (`≈ 3.03 Katha`).
8. **Integrations:** Consumed by `POST /api/sales` (single, multi-plot, whole-project) and `plot-inventory` report.
9. **Code Map:** `src/app/projects/plots/page.tsx` , `src/lib/tenantRules.ts` (`sellPlot`), `src/lib/area.ts` .
10. **Edge Cases & Validations:** Multi-plot sales store the first plot in `land_sales.plot_id` while updating all selected plots in `project_plots` .
11. **Known Limitations:** Automated plot-hold expiry timer is not yet scheduled in `src/worker.ts` .

Module 06: Project Expenses

Last Verified: 2026-11-12

1. **Purpose:** Captures project acquisition, legal, development, and overhead expenses with approval workflow and 3 payment channels.
2. **Who Uses It:** Site Manager, Accountant, Managing Partner, Owner (`expenses.view` , `expenses.create` , `expenses.approve` , `expenses.delete`).
3. **Screens & Routes:** `/expenses` , `/expenses/dashboard` , `/expenses/approvals` , `/expenses/breakdown` .
4. **Main Workflow:**

MERMAID DIAGRAM

```
flowchart LR
  Post["POST /api/expenses (PENDING)"] --> App["POST /api/expenses/[id]/approve"]
  App -->|CASH/BANK| TL["treasury_ledger"]
  App -->|PAID_BY_PARTNER| PT["partner_transactions"]
```

5. **Data (Tables Read & Written):** Reads/Writes `project_expenses` , `treasury_ledger` , `bank_accounts` , `partner_transactions` , `audit_log` .
6. **Business Rules:** `PAID_BY_PARTNER` never touches `treasury_ledger` (`BR-01`); only `APPROVED` expenses count in PnL (`BR-02`). Validates category, positive amount, and channel requirements.
7. **Calculations: Worked Example:** `৳ 85,000` (`DEVELOPMENT` , `CASH`) approved -> `project_expenses.status = APPROVED` , `treasury_ledger` debited `৳ 85,000` .
8. **Integrations:** Linked to `vouchers` UI, `treasury_ledger` , `partner_transactions` , `project-cost` , and `project-pnl` .
9. **Code Map:** `src/app/expenses/**` , `src/app/api/expenses/route.ts` , `src/app/api/expenses/[id]/approve/route.ts` .
10. **Edge Cases & Validations:** Supports both snake_case and camelCase FormData keys and returns `400` JSON on invalid/missing fields.
11. **Known Limitations:** Uploaded voucher files are saved locally without strict MIME allowlist checks.

Module 07: Voucher Management

Last Verified: 2026-11-12

1. **Purpose:** Provides centralized voucher listing, approval actions, and printable voucher retrieval for expenses.
2. **Who Uses It:** Accountant, Managing Partner, Owner (`expenses.view` , `expenses.approve`).
3. **Screens & Routes:** `/vouchers` .
4. **Main Workflow:**

MERMAID DIAGRAM

```
flowchart LR
  List["/vouchers (GET /api/vouchers)"] --> Action["POST /api/vouchers/[id]/action"]
  List --> Print["GET /api/expenses/[id]/voucher"]
```

5. **Data (Tables Read & Written):** Reads/Writes `project_expenses` , `audit_log` (and treasury/partner ledgers on approval).
6. **Business Rules:** Voucher approvals follow the same `BR-01` and `BR-02` rules as `/expenses/approvals` .
7. **Calculations:** N/A (delegates ledger math to expense approval).
8. **Integrations:** Backed by `project_expenses` and `src/lib/storage.ts` for attached receipt images.
9. **Code Map:** `src/app/vouchers/page.tsx` , `src/app/api/vouchers/**` , `src/app/api/expenses/[id]/voucher/route.ts` .
10. **Edge Cases & Validations:** Expenses without an uploaded voucher image still generate a printable system voucher.
11. **Known Limitations:** Bulk multi-voucher approval in one click is not implemented.

Module 08: Plot Sales & Dues

Last Verified: 2026-11-12

1. **Purpose:** Handles single-plot, multi-plot, and whole-project sales, installment schedules, FIFO collections, overpayment guards, and money receipts.
2. **Who Uses It:** Owner, Managing Partner, Accountant (`sales.view` , `sales.create` , `sales.collect` , `receipts.print`).
3. **Screens & Routes:** `/sales` , `/sales/bookings` , `/sales/dues` , `/sales/[id]` .
4. **Main Workflow:**

MERMAID DIAGRAM

```
flowchart LR
  Create["POST /api/sales"] --> Guard{"advance <= total?"}
  Guard -->|Yes| Sale["land_sales + installment_schedule"]
  Pay["POST /api/sales/[id]/pay"] --> FIFO["collectPayment() FIFO"]
```

5. **Data (Tables Read & Written):** Reads/Writes `land_sales` , `project_plots` , `installment_schedule` , `customer_payments` , `treasury_ledger` , `bank_accounts` , `audit_log` .
6. **Business Rules:** Advance <= total price (`BR-06`), payment <= `currentDueAmount` (`BR-07`), FIFO installment settlement (`BR-08`), full payment sets `PAID_IN_FULL` & `SOLD` (`BR-09`).
7. **Calculations: Worked Example:** Price `৳ 12,00,000` , Advance `৳ 2,00,000` , 4 installments -> `current_due_amount = ৳ 10,00,000` (`৳ 2,50,000` each). Paying `৳ 2,50,000` settles Installment #1 (`PAID`) and lowers due to `৳ 7,50,000` .
8. **Integrations:** Credits `treasury_ledger` / `bank_accounts` and feeds Sales/Collection/Receivables reports.
9. **Code Map:** `src/app/sales/` , **`src/app/api/sales/`** , `src/lib/collections.ts` , `src/lib/tenantRules.ts` .
10. **Edge Cases & Validations:** Bank/Cheque/MFS advance payments validate that a valid `bank_account` is selected.
11. **Known Limitations:** Multi-plot sales link `land_sales.plot_id` to the first plot; individual secondary plot IDs are tracked via `project_plots.status` & `audit_log` .

Module 09: Reports & Profit Distribution

Last Verified: 2026-11-12

1. **Purpose:** Provides 26 financial, partner, project, sales, profit, and audit reports with Grid/List catalog views, Fiscal Year filtering, and PDF/Excel export.
2. **Who Uses It:** Role-gated per category (`reports.view` , `reports.export` , `reports.*.view` , `profit.distribute`).
3. **Screens & Routes:** `/reports` (catalog with category filters & Grid/List toggle), `/reports/[key]` , `/reports/project-pnl` .
4. **Main Workflow:**

MERMAID DIAGRAM

```
flowchart LR
  Cat["/reports"] --> Detail["/reports/[key]?preset=this_fy"]
  Detail --> FY["Resolve active FY (fiscalYear.ts)"]
  Detail --> Export["/api/reports/[key]?format=pdf|excel"]
```

5. **Data (Tables Read & Written):** Reads all financial/operational tables; writes `profit_distributions` , `partner_transactions` , `audit_log` on profit distribution.
6. **Business Rules:** Only `APPROVED` expenses count (`BR-02`); `this_fy` uses tenant's active FY from `dashboard_settings` when custom dates aren't passed.
7. **Calculations: Worked Example:** Revenue `৳ 50,00,000` - Approved Expenses `৳ 35,00,000` = Net Profit `৳ 15,00,000` . Partner (60%) = `৳ 9,00,000` ; Partner (40%) = `৳ 6,00,000` .
8. **Integrations:** Shares query logic with Dashboard widgets; exports via Puppeteer (`pdf.ts`) and Excel HTML (`excel.ts`).
9. **Code Map:** `src/app/reports/` , **`src/components/reports/`** , `src/lib/reports/` , **`src/app/api/reports/`** .
10. **Edge Cases & Validations:** Switching preset in `ReportFilters.tsx` omits stale `from` / `to` dates unless `custom` is selected.
11. **Known Limitations:** PDF export requires Chromium via Puppeteer; falls back to HTML if Chromium fails to launch.

Module 10: Audit Trail

Last Verified: 2026-11-12

1. **Purpose:** Displays an immutable, filterable log of every create, update, approval, and settings change with actor attribution.
2. **Who Uses It:** Owner, Managing Partner, Auditors (`audit.view`, `reports.audit.view`).
3. **Screens & Routes:** `/audit`, `/reports/audit-trail`, `/reports/user-activity`, `/reports/reconciliation`.
4. **Main Workflow:**

MERMAID DIAGRAM

```
flowchart LR
  Mut["Any Financial/Settings Mutation"] --> WA["writeAudit()"]
  WA --> AL["audit_log (old_values, new_values, changed_by)"]
```

5. **Data (Tables Read & Written):** Reads `audit_log` and `control_plane.global_users` (actor names/emails).
6. **Business Rules:** Every financial write must create an `audit_log` row with non-null `changed_by` (`BR-10`).
7. **Calculations:** N/A (append-only event log).
8. **Integrations:** Called inside `$transaction` across expenses, sales, payments, contra transfers, profit distribution, and Fiscal Years.
9. **Code Map:** `src/app/audit/page.tsx`, `src/lib/tenantRules.ts` (`writeAudit`), `src/lib/reports/audit-trail.ts`.
10. **Edge Cases & Validations:** Reconciliation report cross-checks ledger sums against cached balances.
11. **Known Limitations:** Control-plane `audit_logs` are separate from tenant `audit_log`.

Module 11: Users & Invitations

Last Verified: 2026-11-12

1. **Purpose:** Allows administrators to create staff accounts, send invite tokens, assign roles, reset passwords, and deactivate users.
2. **Who Uses It:** Owner (`users.view` , `users.manage`).
3. **Screens & Routes:** `/settings/users` , `/invite/[token]` .
4. **Main Workflow:**

MERMAID DIAGRAM

```
flowchart LR
  Admin["/settings/users"] -->|Direct| Create["POST /api/users (mustChangePassword=true)"]
  Admin -->|Invite| Inv["POST /api/invites -> /invite/[token]"]
```

5. **Data (Tables Read & Written):** Reads/Writes `control_plane.global_users` , `control_plane.user_invitations` , `tenant.user_roles` , `tenant.roles` .
6. **Business Rules:** Self-lockout protection prevents admins from deactivating themselves; deactivated users lose session access immediately.
7. **Calculations:** N/A.
8. **Integrations:** Links `global_users.id` to `tenant.user_roles.user_id` .
9. **Code Map:** `src/app/settings/users/` , **`src/app/api/users/`** , `src/app/api/invites/**` , `src/app/invite/[token]/page.tsx` .
10. **Edge Cases & Validations:** Invite acceptance atomically marks `user_invitations.acceptedAt` and creates `global_users` + `user_roles` .
11. **Known Limitations:** Invite links must be copied/shared manually unless an email sender is configured.

Module 12: Roles & Permissions

Last Verified: 2026-11-12

1. **Purpose:** Manages tenant RBAC roles, permission assignments across 79 keys, and role duplication.
2. **Who Uses It:** Owner (`roles.manage`).
3. **Screens & Routes:** `/settings/roles` .
4. **Main Workflow:**

MERMAID DIAGRAM

```
flowchart LR
  UI["/settings/roles"] --> API["/api/roles & /api/roles/[id]"]
  API --> DB["roles + role_permissions"]
```

5. **Data (Tables Read & Written):** Reads/Writes `roles` , `role_permissions` , `user_roles` .
6. **Business Rules:** System `Owner` role cannot be edited or deleted (`BR-14`); changes take effect immediately on next request.
7. **Calculations:** N/A.
8. **Integrations:** Evaluated by `can()` in `src/lib/rbac.ts` across every page, API, and dashboard widget.
9. **Code Map:** `src/app/settings/roles/` , **`src/app/api/roles/`** , `src/lib/permissions.ts` , `src/lib/rbac.ts` , `src/lib/seedRoles.ts` .
10. **Edge Cases & Validations:** Duplicate role endpoint (`/api/roles/[id]/duplicate`) clones permissions into a non-system custom role.
11. **Known Limitations:** Permission keys are code-defined in `src/lib/permissions.ts` .

Module 13: Executive Dashboard & Fiscal Years (FY)

Last Verified: 2026-11-12

1. **Purpose:** Renders permission-gated executive KPIs/charts across 6 tabs and lets admins configure Fiscal Years and the active reporting FY.
2. **Who Uses It:** All authenticated staff (`dashboard.view` , `dashboard.tab.` , *`dashboard.widget.`*); FY mutations require `dashboard.settings.manage` , `roles.manage` , or `users.manage` .
3. **Screens & Routes:** `/dashboard` , `/settings/fiscal-years` .
4. **Main Workflow:**

MERMAID DIAGRAM

```
flowchart LR
  Dash["/dashboard"] --> Layout["/api/dashboard/layout + /api/dashboard/widgets/[key]"]
  FY["/settings/fiscal-years"] --> Settings["dashboard_settings (fiscal_years)"]
  Settings -->|fy-updated| Header["AppShell Header FY Badge"]
```

5. **Data (Tables Read & Written):** Reads/Writes `dashboard_layouts` , `dashboard_settings` , `audit_log` .
6. **Business Rules:** Current active FY cannot be deleted (`BR-14`); duplicate FY labels and `endDate < startDate` are rejected with `400` .
7. **Calculations: Worked Example:** BD FY `2026-2027` runs `2026-07-01` to `2027-06-30` ; switching active FY updates `AppShell` header badge and `preset=this_fy` reports.
8. **Integrations:** Drives `preset=this_fy` date filtering in `/reports/[key]` and `/api/reports/[key]` .
9. **Code Map:** `src/app/dashboard/` **,** `src/lib/dashboard/` , `src/lib/fiscalYear.ts` , `src/app/settings/fiscal-years/**` , `src/app/api/settings/fiscal-years/route.ts` .
10. **Edge Cases & Validations:** If `fiscal_years` has not been customized yet, `getFiscalYears()` returns default BD Jul–Jun current & previous FYs.
11. **Known Limitations:** Widget data queries assume a single active FY badge at a time.

Module 14: SaaS Billing

Last Verified: 2026-11-12

1. **Purpose:** Tracks tenant subscription plans and invoices in the control plane.
2. **Who Uses It:** Owner (`billing.manage`).
3. **Screens & Routes:** `/settings/billing` .
4. **Main Workflow:**

MERMAID DIAGRAM

```
flowchart LR
  Inv["GET /api/billing/invoices"] --> Pay["POST /api/billing/pay (Stub)"]
  Pay --> CDB["control_plane.billing_invoices (PAID)"]
```

5. **Data (Tables Read & Written):** Reads/Writes `control_plane.billing_invoices` , `control_plane.subscription_plans` , `control_plane.tenants` .
6. **Business Rules:** Restricted to `billing.manage` (Owner role).
7. **Calculations:** Plan price in BDT (`subscription_plans.price_bdt`) per billing cycle.
8. **Integrations:** Reads `tenantId` from session JWT.
9. **Code Map:** `src/app/settings/billing/**` , `src/app/api/billing/invoices/route.ts` , `src/app/api/billing/pay/route.ts` .
10. **Edge Cases & Validations:** Non-owners receive `403 Forbidden` .
11. **Known Limitations: Stubbed:** `/api/billing/pay` simulates payment without a live bKash/SSLCommerz/Stripe gateway.

---

06. End-to-End Business Workflows

Purpose: Step-by-step explanations and Mermaid diagrams for all core LandInvest workflows.

Intended Reader: Business Owner, Technical Lead, QA Engineer, New Developer.

Last Verified: 2026-11-12

---

1. Tenant Signup & Provisioning

1. User submits organization name, email, and password at `/signup` (`POST /api/signup`).
2. API creates a `tenants` row (`status = "trial"`, unique `db_name`) and `global_users` row (`isOwner = true`), then enqueues a job on BullMQ queue `tenant-provisioning`.
3. `src/worker.ts` picks up the job, runs `CREATE DATABASE`, pushes `prisma/tenant/schema.prisma`, seeds `default_categories`, seeds 4 default roles, and assigns `Owner` to the signup user.

MERMAID DIAGRAM

```
sequenceDiagram
  actor Owner
  participant API as POST /api/signup
  participant CDB as control_plane
  participant Q as Redis (tenant-provisioning)
  participant W as src/worker.ts
  participant TDB as tenant_<slug>

  Owner->>API: Submit org & owner credentials
  API->>CDB: Insert Tenant + GlobalUser (isOwner=1)
  API->>Q: Enqueue provisioning job
  API-->>Owner: 200 Queued
  W->>Q: Consume job
  W->>TDB: CREATE DATABASE + prisma db push
  W->>TDB: Seed default_categories + 4 roles + assign Owner
```

---

2. Login, Session & Tenant Resolution

1. User submits email + password at `/login` (`next-auth` CredentialsProvider).
2. `src/auth.ts` verifies `global_users.passwordHash` (`bcryptjs`), `isActive === true`, and `tenant.status` in `["active", "trial"]`.
3. JWT stores `tenantId`, `user_id`, `isOwner`, `dbName`. Subsequent requests call `getTenantClientFromSession()` to resolve the cached `PrismaClient` for that tenant.

---

3. User Invitation & Direct Creation

- **Direct Creation:** Admin creates user via `POST /api/users` with role + temporary password (`mustChangePassword = true`).
- **Invitation:** Admin sends invite via `POST /api/invites` creating a `user_invitations` token; invitee accepts at `/invite/[token]`.

---

4. Capital Pooling (Partner Deposit / Withdrawal)

1. Manager/Owner records a deposit or withdrawal on `/partners/capital`.
2. Within `$transaction`: writes `partner_transactions`, writes `treasury_ledger` (`CASH` or `BANK`), updates `bank_accounts.current_balance` if `BANK`, and records `audit_log`.

MERMAID DIAGRAM

```
flowchart LR
  P["Partner Capital Form"] --> TX["Prisma $transaction"]
  TX --> PT["partner_transactions\n(DEPOSIT / WITHDRAWAL)"]
  TX --> TL["treasury_ledger\n(CASH or BANK)"]
  TL -->|If BANK| BA["bank_accounts.current_balance"]
  TX --> AL["audit_log"]
```

---

5. Land Acquisition & 6. Project Lifecycle State Machine

1. User completes the Land Acquisition wizard (`/projects/new`): project details, Mouza/JL/Dag/Khatian metadata, partner share % (must sum to 100%), and optional placeholder plot generation.
2. Project transitions across lifecycle states:

MERMAID DIAGRAM

```
stateDiagram-v2
  [*] --> ACQUIRING: Project Created (Bayna / Acquisition)
  ACQUIRING --> ACTIVE: Acquisition Approved / Land Registered
  ACTIVE --> COMPLETED: All Plots Sold & Profit Distributed
  ACQUIRING --> ON_HOLD: Paused
  ACTIVE --> ON_HOLD: Paused
  ON_HOLD --> ACTIVE: Resumed
```

---

7. Expense Voucher Approval & Three Payment Channels

1. User creates an expense (`POST /api/expenses`) -> status `PENDING` + `audit_log`.
2. Approver clicks Approve/Reject (`POST /api/expenses/[id]/approve`):

- **CASH:** Writes `treasury_ledger` (`CASH`, `DEBIT`).
- **BANK:** Writes `treasury_ledger` (`BANK`, `DEBIT`) and decrements `bank_accounts.current_balance`.
- **PAID_BY_PARTNER:** Never touches `treasury_ledger`; writes `partner_transactions` (`DIRECT_EXPENSE`)!

MERMAID DIAGRAM

```
flowchart TD
  Create["POST /api/expenses -> status: PENDING"] --> Decision{"POST /api/expenses/[id]/approve"}
  Decision -->|REJECTED| Rej["Update status=REJECTED + audit_log"]
  Decision -->|APPROVED| Channel{"payment_channel?"}
  Channel -->|CASH| Cash["Write treasury_ledger (CASH DEBIT) + audit_log"]
  Channel -->|BANK| Bank["Write treasury_ledger (BANK DEBIT) + decrement bank_accounts + audit_log"]
  Channel -->|PAID_BY_PARTNER| Partner["Write partner_transactions (DIRECT_EXPENSE) + audit_log\n(NO treasury_
```

---

8. Plot Status State Machine

MERMAID DIAGRAM

```
stateDiagram-v2
  [*] --> AVAILABLE: Plot Created / Generated
  AVAILABLE --> BOOKED: Partial / Installment Sale Created
  AVAILABLE --> SOLD: 100% Upfront Payment Sale Created
  BOOKED --> SOLD: Installments Settled (current_due_amount == 0)
```

9. Plot Sale, Installments & Collections

1. Sale created via `POST /api/sales` (single plot, multi-plot `plot_ids`, or `is_whole_project`). Validates `advance <= total_agreed_price`.
2. Creates `land_sales` (`PAID_IN_FULL`, `PARTIAL`, or `BOOKED`), updates `project_plots` (`SOLD` or `BOOKED`), generates `installment_schedule` rows, and records advance in `customer_payments` + `treasury_ledger`.
3. Subsequent payments (`POST /api/sales/[id]/pay` via `collectPayment()` in `src/lib/collections.ts`) validate `amount <= currentDueAmount`, settle `installment_schedule` rows **FIFO**, update `land_sales.status` to `PAID_IN_FULL` and plots to `SOLD` when due reaches `0`.

MERMAID DIAGRAM

```
flowchart LR
  Sale["POST /api/sales"] -->|Advance > 0| PayAdv["customer_payments + treasury_ledger"]
  Sale -->|Installments > 0| Sched["installment_schedule (UPCOMING)"]
  Collect["POST /api/sales/[id]/pay"] --> Guard{"amount <= currentDueAmount?"}
  Guard -->|No| Err["400 Error"]
  Guard -->|Yes| FIFO["Settle installment_schedule FIFO\n+ customer_payments + treasury_ledger"]
  FIFO -->|due == 0| Sold["land_sales=PAID_IN_FULL\nproject_plots=SOLD"]
```

10. Contra Transfer (Cash ↔ Bank)

- **Accountant** (`POST /api/treasury/contra` without `treasury.contra.approve`): creates a `PENDING` `treasury_transfer_requests` row.
- **Owner / Managing Partner** (`POST /api/treasury/contra/[id]/approve` or direct creation): writes paired `DEBIT` and `CREDIT` rows in `treasury_ledger`, updates `bank_accounts.current_balance`, and records `audit_log`.

11. Profit Calculation & Distribution

MERMAID DIAGRAM

```
flowchart LR
  Rev["Total Sales Revenue\n(land_sales)"] --> Net["Net Profit =\nRevenue - Approved Project Expenses"]
  Exp["Approved project_expenses"] --> Net
  Net --> Split["Split by project_partners.share_percent\n(Total = 100%)"]
  Split --> Dist["profit_distributions + partner_transactions (PROFIT_CREDIT) + audit_log"]
```

12. Report Generation & Export

1. User opens `/reports` (filterable by category and switchable between Grid/List view).

2. Opens `/reports/[key]`, selects preset (including `this_fy` which resolves the tenant's active Fiscal Year from `dashboard_settings`) or custom dates.
3. Exports via `/api/reports/[key]?format=pdf|excel` using Puppeteer (PDF) or HTML table (Excel).

---

13. Role & Permission Changes

1. Admin edits role permissions in `/settings/roles` (`PATCH /api/roles/[id]`).
2. Because `can()` queries `user_roles -> role_permissions` from the tenant DB on each request, permission changes take effect **immediately** without requiring user re-login.

---

14. Tenant Backup, Restore & Migration Rollout

- **Backup:** `mysqldump` of `control_plane` + each `tenant_*` database.
- **Restore:** `mysql < backup.sql` into target DB.
- **Schema Rollout:** Iterate over all active `tenants.db_name` rows in `control_plane` and execute `prisma db push --schema=prisma/tenant/schema.prisma` with `DATABASE_URL_TENANT` set per tenant.

`;

writeDoc("06-workflows.md", ...)`

---

07. Authentication, Roles & Permissions

Purpose: Documents authentication, JWT sessions, tenant resolution, all 79 permission keys, default roles, and RBAC safety rules.

Intended Reader: Security Reviewer, Technical Lead, Developer, Admin Owner.

Last Verified: 2026-11-12

Authentication & Session Architecture

- **Provider:** Auth.js v4 (`next-auth`) with `CredentialsProvider` (`src/auth.ts`).
- **Passwords:** Hashed with `bcryptjs` . Users invited or created with temporary passwords have `mustChangePassword = true` , enforced by `AppShell.tsx` redirecting to `/change-password` .
- **Session Strategy:** Stateless JWT enriched with `user_id` , `tenantId` , `isOwner` , `mustChangePassword` , `fullName` , and `dbName` .
- **Live Revocation Check:** On every `session()` callback, `src/auth.ts` queries `control_plane.global_users` and `tenants.status` . Deactivated users or suspended/cancelled tenants receive `null` session (fail-closed).
- **Rate Limiting:** Configurable via `AUTH_RATE_LIMIT_MAX` (Redis / in-memory login throttling).

Permission Evaluation Flow

MERMAID DIAGRAM

```
flowchart TD
  Req["Incoming Request (Page / API / Server Action)"] --> Sess["getServerSession(authOptions)"]
  Sess -->|No session| Unauth["401 Unauthorized / Redirect /login"]
  Sess -->|Session exists| OwnerCheck{"user.isOwner === true?"}
  OwnerCheck -->|Yes| Allow["ALLOW (Full Access)"]
  OwnerCheck -->|No| LoadPerms["Load user_roles -> role_permissions from Tenant DB"]
  LoadPerms --> HasKey{"Permission key in set?"}
  HasKey -->|Yes| Allow
  HasKey -->|No| Deny["403 Forbidden / Forbidden UI Marker"]
```

Default Roles (`src/lib/seedRoles.ts`)

Role Name	System Role	Permission Count	Description
Owner	Yes (Protected)	79	Full access — cannot be edited or deleted
Managing Partner	No	76	Executive access across all operations, financial ledgers, and reporting
Accountant	No	27	Treasury ledger, cash & bank management, payment collection, and expense accounting
Site Manager	No	20	Land projects inspection, plots management, and on-site project expense entries

---

Role vs Module Permission Matrix (Base Permissions)

Permission Key	Owner	Managing Partner	Accountant	Site Manager
`dashboard.view`	YES	YES	YES	YES
`dashboard.customize`	YES	YES	YES	YES
`dashboard.layout.manage`	YES	YES	-	-
`dashboard.export`	YES	YES	YES	-
`dashboard.settings.manage`	YES	YES	-	-
`partners.view`	YES	YES	-	-
`partners.manage`	YES	YES	-	-
`partners.capital.deposit`	YES	YES	-	-
`partners.capital.withdraw`	YES	YES	-	-
`partners.ledger.view`	YES	YES	-	-
`treasury.view`	YES	YES	YES	-
`treasury.bank.manage`	YES	YES	YES	-
`treasury.contra.create`	YES	YES	YES	-
`treasury.contra.approve`	YES	YES	-	-
`projects.view`	YES	YES	-	YES
`projects.create`	YES	YES	-	-
`projects.edit`	YES	YES	-	-
`projects.delete`	YES	YES	-	-
`plots.manage`	YES	YES	-	YES
`expenses.view`	YES	YES	YES	YES
`expenses.create`	YES	YES	YES	YES
`expenses.approve`	YES	YES	-	-
`expenses.delete`	YES	YES	-	-
`sales.view`	YES	YES	YES	-
`sales.create`	YES	YES	-	-
`sales.collect`	YES	YES	YES	-
`receipts.print`	YES	YES	YES	-
`profit.view`	YES	YES	-	-
`profit.distribute`	YES	YES	-	-
`reports.view`	YES	YES	YES	YES
`reports.export`	YES	YES	-	-

Permission Key	Owner	Managing Partner	Accountant	Site Manager
`reports.financial.view`	YES	YES	YES	-
`reports.partners.view`	YES	YES	-	-
`reports.projects.view`	YES	YES	-	YES
`reports.sales.view`	YES	YES	-	-
`reports.audit.view`	YES	YES	-	-
`users.view`	YES	YES	-	-
`users.manage`	YES	-	-	-
`roles.manage`	YES	-	-	-
`billing.manage`	YES	-	-	-
`audit.view`	YES	YES	-	-

Complete Permission Catalog (79 Keys across 9 Groups)

Permission Key	Module	English Label	Bengali Label
`dashboard.view`	Dashboard & Executive Views	View dashboard	ড্যাশবোর্ড দেখুন
`dashboard.customize`	Dashboard & Executive Views	Customize own dashboard layout	নিজের ড্যাশবোর্ড কাস্টমাইজ করুন
`dashboard.layout.manage`	Dashboard & Executive Views	Manage role default layouts	রোলের ডিফল্ট লেআউট পরিচালনা করুন
`dashboard.export`	Dashboard & Executive Views	Export dashboard as PDF/PNG	ড্যাশবোর্ড এক্সপোর্ট (পিডিএফ/ছবি)
`dashboard.settings.manage`	Dashboard & Executive Views	Manage dashboard settings & thresholds	ড্যাশবোর্ড সেটিংস ও থ্রেশহোল্ড পরিচালনা
`dashboard.tab.overview`	Dashboard & Executive Views	View Tab: Executive Overview	ট্যাব দেখুন: এক্সিকিউটিভ ওভারভিউ
`dashboard.tab.financial`	Dashboard & Executive Views	View Tab: Treasury & Financials	ট্যাব দেখুন: ট্রেজারি ও আর্থিক অবস্থা
`dashboard.tab.projects`	Dashboard & Executive Views	View Tab: Projects & Land	ট্যাব দেখুন: প্রকল্প ও জমি
`dashboard.tab.sales`	Dashboard & Executive Views	View Tab: Sales & Collections	ট্যাব দেখুন: বিক্রয় ও আদায়
`dashboard.tab.partners`	Dashboard & Executive Views	View Tab: Partners & Capital	ট্যাব দেখুন: অংশীদার ও মূলধন
`dashboard.tab.operations`	Dashboard & Executive Views	View Tab: Operations & Approvals	ট্যাব দেখুন: অপারেশনস ও অনুমোদন
`dashboard.widget.cash_balance`	Dashboard & Executive Views	Widget: Cash in Hand	উইজেট: হাতে নগদ

Permission Key	Module	English Label	Bengali Label
`dashboard.widget.bank_balance`	Dashboard & Executive Views	Widget: Total Bank Balance	উইজেট: মোট ব্যাংক ব্যালেন্স
`dashboard.widget.total_liquidity`	Dashboard & Executive Views	Widget: Total Liquidity	উইজেট: মোট তারল্য
`dashboard.widget.working_capital`	Dashboard & Executive Views	Widget: Available Working Capital	উইজেট: কার্যনির্বাহী মূলধন
`dashboard.widget.partner_equity`	Dashboard & Executive Views	Widget: Partner Equity Injected	উইজেট: অংশীদার মূলধন বিনিয়োগ
`dashboard.widget.capital_deployed`	Dashboard & Executive Views	Widget: Capital Deployed in Land	উইজেট: প্রকল্পে নিয়োজিত মূলধন
`dashboard.widget.unsold_inventory`	Dashboard & Executive Views	Widget: Unsold Inventory Value	উইজেট: অবিক্রীত ইনভেন্টরি মূল্য
`dashboard.widget.total_receivables`	Dashboard & Executive Views	Widget: Total Receivables	উইজেট: মোট বকেয়া/প্রাপ্য
`dashboard.widget.overdue_receivables`	Dashboard & Executive Views	Widget: Overdue Receivables	উইজেট: মেয়াদোত্তীর্ণ বকেয়া
`dashboard.widget.sales_month`	Dashboard & Executive Views	Widget: Sales This Month	উইজেট: চলতি মাসের বিক্রয়
`dashboard.widget.collections_month`	Dashboard & Executive Views	Widget: Collections This Month	উইজেট: চলতি মাসের নগদ আদায়
`dashboard.widget.expenses_month`	Dashboard & Executive Views	Widget: Expenses This Month	উইজেট: চলতি মাসের খরচ
`dashboard.widget.pending_approvals`	Dashboard & Executive Views	Widget: Pending Approvals	উইজেট: অনুমোদনাধীন ভাউচার
`dashboard.widget.net_profit`	Dashboard & Executive Views	Widget: Net Profit & Realized Cash	উইজেট: নিট মুনাফা ও বাস্তবায়িত লাভ
`dashboard.widget.portfolio_roi`	Dashboard & Executive Views	Widget: Portfolio ROI %	উইজেট: পোর্টফোলিও আরওআই %
`dashboard.widget.cash_flow_trend`	Dashboard & Executive Views	Widget: 12-Month Cash Flow Trend	উইজেট: ১২ মাসের ক্যাশ ফ্লো ট্রেন্ড
`dashboard.widget.project_cost_vs_sales`	Dashboard & Executive Views	Widget: Project Cost vs Sales vs Collected	উইজেট: প্রকল্প খরচ বনাম বিক্রয় বনাম আদায়
`dashboard.widget.project_cost_share`	Dashboard & Executive Views	Widget: Project-wise Cost Share	উইজেট: প্রকল্পভিত্তিক খরচ বণ্টন
`dashboard.widget.expense_breakdown`	Dashboard & Executive Views	Widget: Expense Breakdown by Category	উইজেট: খাতভিত্তিক ব্যয়ের বিভাজন
`dashboard.widget.receivables_aging`	Dashboard & Executive Views	Widget: Receivables Aging	উইজেট: বকেয়া এইজিং বিশ্লেষণ
`dashboard.widget.plot_inventory_status`	Dashboard & Executive Views	Widget: Plot Inventory Status	উইজেট: প্লট ইনভেন্টরি অবস্থা

Permission Key	Module	English Label	Bengali Label
`dashboard.widget.partner_equity_dist`	Dashboard & Executive Views	Widget: Partner Equity Distribution	উইজেট: অংশীদারদের মূলধন বণ্টন
`dashboard.widget.profit_by_project`	Dashboard & Executive Views	Widget: Profit by Project	উইজেট: প্রকল্পভিত্তিক নিট মুনাফা
`dashboard.widget.collection_trend`	Dashboard & Executive Views	Widget: Collection Trend vs Scheduled Dues	উইজেট: আদায় ট্রেন্ড বনাম নির্ধারিত বকেয়া
`dashboard.widget.cash_flow_forecast`	Dashboard & Executive Views	Widget: 90-Day Cash Flow Forecast	উইজেট: ৯০ দিনের ক্যাশ ফ্লো পূর্বাভাস
`dashboard.widget.recent_transactions`	Dashboard & Executive Views	Widget: Recent Treasury Transactions	উইজেট: সাম্প্রতিক ট্রেজারি লেনদেন
`dashboard.widget.overdue_installments`	Dashboard & Executive Views	Widget: Overdue Installments List	উইজেট: মেয়াদোত্তীর্ণ কিস্তির তালিকা
`dashboard.widget.project_summary`	Dashboard & Executive Views	Widget: Projects Summary Table	উইজেট: প্রকল্পের সংক্ষিপ্ত বিবরণী
`dashboard.widget.partner_summary`	Dashboard & Executive Views	Widget: Partners Capital & Balance	উইজেট: অংশীদারদের মূলধন ও বর্তমান স্থিতি
`dashboard.widget.pending_vouchers`	Dashboard & Executive Views	Widget: Pending Expense Vouchers	উইজেট: অনুমোদনাধীন ব্যয়ের তালিকা
`dashboard.widget.quick_actions`	Dashboard & Executive Views	Widget: Quick Actions	উইজেট: দ্রুত অ্যাকশন
`dashboard.widget.active_alerts`	Dashboard & Executive Views	Widget: Smart Insights & Alerts	উইজেট: স্মার্ট ইনসাইট ও সতর্কবার্তা
`partners.view`	Partners	View partners	পার্টনার দেখুন
`partners.manage`	Partners	Manage partners	পার্টনার ব্যবস্থাপনা
`partners.capital.deposit`	Partners	Record capital deposit	মূলধন জমা
`partners.capital.withdraw`	Partners	Record capital withdrawal	মূলধন উত্তোলন
`partners.ledger.view`	Partners	View partner ledger	পার্টনার লেজার দেখুন
`treasury.view`	Treasury	View treasury	ট্রেজারি দেখুন
`treasury.bank.manage`	Treasury	Manage bank accounts	ব্যাংক অ্যাকাউন্ট ব্যবস্থাপনা
`treasury.contra.create`	Treasury	Create contra transfer	কন্ট্রা তৈরি
`treasury.contra.approve`	Treasury	Approve contra transfer	কন্ট্রা অনুমোদন
`projects.view`	Land Projects	View projects	প্রকল্প দেখুন
`projects.create`	Land Projects	Create project	প্রকল্প তৈরি
`projects.edit`	Land Projects	Edit project	প্রকল্প সম্পাদনা
`projects.delete`	Land Projects	Delete project / plot	প্রকল্প/প্লট মুছুন
`plots.manage`	Land Projects	Manage plots	প্লট ব্যবস্থাপনা
`expenses.view`	Expenses	View expenses	ব্যয় দেখুন

Permission Key	Module	English Label	Bengali Label
`expenses.create`	Expenses	Create expense	ব্যয় তৈরি
`expenses.approve`	Expenses	Approve expense	ব্যয় অনুমোদন
`expenses.delete`	Expenses	Delete expense	ব্যয় মুছুন
`sales.view`	Sales	View sales	বিক্রয় দেখুন
`sales.create`	Sales	Create sale / booking	বিক্রয় তৈরি
`sales.collect`	Sales	Collect payment	পেমেন্ট আদায়
`receipts.print`	Sales	Print receipts	রসিদ প্রিন্ট
`profit.view`	Profit	View PnL	লাভ-ক্ষতি দেখুন
`profit.distribute`	Profit	Distribute profit	লাভ বণ্টন
`reports.view`	Reports	View reports	রিপোর্ট দেখুন
`reports.export`	Reports	Export reports	রিপোর্ট এক্সপোর্ট
`reports.financial.view`	Reports	View financial reports	আর্থিক প্রতিবেদন দেখুন
`reports.partners.view`	Reports	View partner reports	অংশীদার প্রতিবেদন দেখুন
`reports.projects.view`	Reports	View project reports	প্রকল্প প্রতিবেদন দেখুন
`reports.sales.view`	Reports	View sales reports	বিক্রয় প্রতিবেদন দেখুন
`reports.audit.view`	Reports	View audit reports	অডিট প্রতিবেদন দেখুন
`users.view`	Administration	View users	ইউজার দেখুন
`users.manage`	Administration	Manage users	ইউজার ব্যবস্থাপনা
`roles.manage`	Administration	Manage roles	রোল ব্যবস্থাপনা
`billing.manage`	Administration	Manage billing	বিলিং ব্যবস্থাপনা
`audit.view`	Administration	View audit log	অডিট লগ দেখুন

RBAC Safety Rules

1. **Owner Protection:** The `Owner` role (`isSystem: true`) cannot be edited or deleted via `/api/roles/[id]` .
2. **Privilege Escalation Prevention:** Non-owners cannot grant permissions they do not hold or assign the `Owner` role.
3. **Self-Lockout Prevention:** Users cannot deactivate themselves or strip `users.manage` / `roles.manage` from their own active role.

---

08. Business Rules Register

Purpose: Numbered register of all enforced business rules, why they exist, where they are enforced in code, and which tests verify them.

Intended Reader: Technical Lead, QA Engineer, Business Analyst, New Developer.

Last Verified: 2026-11-12

#	Rule Statement	Why It Exists	Enforced In	Verified By
BR-01	**Paid-by-Partner expenses never touch `treasury_ledger`**	Partner-paid expenses do not leave company cash or bank; they credit the partner's ledger (`DIRECT_EXPENSE`) instead.	`src/lib/tenantRules.ts` (`isPaidByPartner`), `src/app/api/expenses/[id]/approve/route.ts`	`scripts/acceptance.mjs` (C3 `leaked=0`)
BR-02	**Only `APPROVED` expenses count toward project cost & PnL**	Pending or rejected vouchers must not distort financial statements or profit calculations.	`src/lib/reports/project-cost.ts`, `src/lib/reports/project-pnl.ts`, `src/app/api/reports/pnl/route.ts`	`scripts/acceptance.mjs` (C2 PnL)
BR-03	**Partner ownership shares per project must total 100%**	Prevents unallocated or >100% profit distribution splits.	`src/app/projects/actions.ts`, `src/app/projects/new/page.tsx`	Manual / UI validation
BR-04	**Plot sizes cannot exceed saleable land area**	Prevents overselling subdivided Shotok beyond the acquired project area.	`src/app/projects/[id]/` plot creation actions	UI / Server Action validation
BR-05	**Sold/Booked plots are locked from resale**	Prevents double-selling an already booked or sold plot.	`src/lib/tenantRules.ts` (`sellPlot`), `src/app/api/sales/route.ts`	`scripts/acceptance.mjs` (C2)
BR-06	**Advance payment cannot exceed total agreed price**	Prevents negative due balances on sale creation.	`src/app/api/sales/route.ts` (`advance > totalPrice`)	`scripts/test_full_project.mjs` (Test 6a)
BR-07	**Installment collection cannot exceed `currentDueAmount`**	Prevents overpaying a sale contract beyond remaining receivable.	`src/lib/collections.ts` (`collectPayment`)	`scripts/test_full_project.mjs` (Test 6c)
BR-08	**Installments settle FIFO (oldest due date first)**	Standardizes installment tracking when partial or lump-sum payments arrive.	`src/lib/collections.ts` (`collectPayment`)	`scripts/acceptance.mjs` (C2 FIFO)
BR-09	**Full payment flips sale to `PAID_IN_FULL` and plots to `SOLD`**	Keeps inventory and contract status synchronized upon settlement (upfront or final installment).	`src/lib/collections.ts`, `src/app/api/sales/route.ts`	`scripts/test_full_project.mjs` (Test 6e)

#	Rule Statement	Why It Exists	Enforced In	Verified By
BR-10	**Every financial & settings write emits an `audit_log` row with actor ID**	Guarantees full traceability of who created, approved, or modified any record.	`src/lib/tenantRules.ts` (`writeAudit`), `src/lib/fiscalYear.ts`	`scripts/acceptance.mjs` (C3)
BR-11	**Cash & Bank balances are ledger-derived**	`treasury_ledger` is the single source of truth; `bank_accounts.current_balance` is updated atomically inside `$transaction` .	`src/lib/treasury.ts` , `src/lib/collections.ts`	`scripts/acceptance.mjs` (C2/C3)
BR-12	**Area conversions use Shotok as base unit**	Avoids floating-point drift across Shotok, Katha, and Bigha inputs.	`src/lib/area.ts`	Unit helpers
BR-13	**Non-approver contra transfers require approval**	Accountants can queue Cash ↔ Bank transfers, but only approvers (`treasury.contra.approve`) can post to `treasury_ledger` .	`src/lib/treasury.ts` (`createContra`), `src/app/api/treasury/contra/[id]/approve/route.ts`	`scripts/acceptance.mjs` (C2), `scripts/test_full_project.mjs` (Test 4)
BR-14	**Owner role cannot be edited/deleted; current active FY cannot be deleted**	Prevents system lockout and ensures an active reporting FY always exists.	`src/app/api/roles/[id]/route.ts` , `src/lib/fiscalYear.ts`	`scripts/test_fiscal_years.mjs` (Test 3)

`;

writeDoc("08-business-rules.md", ...)`

---

09. API Routes & Server Actions Inventory

Purpose: Auto-extracted inventory of all Next.js App Router pages, API endpoints, and Server Action files with their detected permission checks.

Intended Reader: Backend/Frontend Developers, Security Auditors.

Last Verified: 2026-11-12

Summary Counts

- **App Pages:** 37
- **API Route Handlers:** 39
- **Server Action Files:** 11

API Endpoints (39)

Route	Methods	Detected Permission Check(s)	File
`/api/auth/change-password`	POST	**None / Public or Session-only**	`src/app/api/auth/change-password/route.ts`
`/api/auth/forgot-password`	POST	**None / Public or Session-only**	`src/app/api/auth/forgot-password/route.ts`
`/api/auth/reset-password`	GET, POST	**None / Public or Session-only**	`src/app/api/auth/reset-password/route.ts`
`/api/auth/[...nextauth]`	HANDLER	**None / Public or Session-only**	`src/app/api/auth/[...nextauth]/route.ts`
`/api/banks`	GET	**None / Public or Session-only**	`src/app/api/banks/route.ts`
`/api/billing/invoices`	GET	`billing.manage`	`src/app/api/billing/invoices/route.ts`
`/api/billing/pay`	POST	`billing.manage`	`src/app/api/billing/pay/route.ts`
`/api/dashboard/insights`	GET	`dashboard.view`	`src/app/api/dashboard/insights/route.ts`
`/api/dashboard/layout`	GET, POST	`dashboard.view`, `dashboard.layout.manage`, `dashboard.customize`	`src/app/api/dashboard/layout/route.ts`
`/api/dashboard/widgets/[key]`	GET	**None / Public or Session-only**	`src/app/api/dashboard/widgets/[key]/route.ts`
`/api/expenses`	GET, POST	`expenses.view`, `expenses.create`	`src/app/api/expenses/route.ts`
`/api/expenses/[id]/approve`	POST	`expenses.approve`	`src/app/api/expenses/[id]/approve/route.ts`

Route	Methods	Detected Permission Check(s)	File
`/api/expenses/[id]/voucher`	GET	`expenses.view`	`src/app/api/expenses/[id]/voucher/route.ts`
`/api/invites/accept`	POST	**None / Public or Session-only**	`src/app/api/invites/accept/route.ts`
`/api/invites`	GET, POST	`users.manage`, `users.view`	`src/app/api/invites/route.ts`
`/api/me/permissions`	GET	**None / Public or Session-only**	`src/app/api/me/permissions/route.ts`
`/api/partners`	GET	**None / Public or Session-only**	`src/app/api/partners/route.ts`
`/api/projects`	GET	**None / Public or Session-only**	`src/app/api/projects/route.ts`
`/api/reports/pnl/distribute`	POST	`profit.distribute`	`src/app/api/reports/pnl/distribute/route.ts`
`/api/reports/pnl/excel`	GET	`dashboard.view`	`src/app/api/reports/pnl/excel/route.ts`
`/api/reports/pnl/pdf`	GET	`dashboard.view`	`src/app/api/reports/pnl/pdf/route.ts`
`/api/reports/pnl`	GET	`profit.view`	`src/app/api/reports/pnl/route.ts`
`/api/reports/[key]`	GET	`reports.export`	`src/app/api/reports/[key]/route.ts`
`/api/roles`	GET, POST	`roles.manage`	`src/app/api/roles/route.ts`
`/api/roles/[id]/duplicate`	POST	`roles.manage`	`src/app/api/roles/[id]/duplicate/route.ts`
`/api/roles/[id]`	PATCH, DELETE	`roles.manage`	`src/app/api/roles/[id]/route.ts`
`/api/sales`	GET, POST	`sales.view`, `sales.create`	`src/app/api/sales/route.ts`
`/api/sales/[id]/pay`	POST	`sales.view`	`src/app/api/sales/[id]/pay/route.ts`
`/api/sales/[id]/receipt`	GET	`sales.view`	`src/app/api/sales/[id]/receipt/route.ts`
`/api/settings/fiscal-years`	GET, POST	`dashboard.settings.manage`, `roles.manage`, `users.manage`	`src/app/api/settings/fiscal-years/route.ts`
`/api/signup`	POST	**None / Public or Session-only**	`src/app/api/signup/route.ts`
`/api/treasury/contra`	GET, POST	`treasury.contra.create`, `treasury.contra.approve`, `treasury.view`	`src/app/api/treasury/contra/route.ts`
`/api/treasury/contra/[id]/approve`	POST	`treasury.contra.approve`	`src/app/api/treasury/contra/[id]/approve/route.ts`

Route	Methods	Detected Permission Check(s)	File
`/api/users`	GET, POST	`users.view`, `users.manage`	`src/app/api/users/route.ts`
`/api/users/[id]/reset`	POST	`users.manage`	`src/app/api/users/[id]/reset/route.ts`
`/api/users/[id]`	PATCH, DELETE	`users.manage`	`src/app/api/users/[id]/route.ts`
`/api/vouchers`	GET	`expenses.view`	`src/app/api/vouchers/route.ts`
`/api/vouchers/[id]/action`	POST	`expenses.approve`	`src/app/api/vouchers/[id]/action/route.ts`
`/api/vouchers/[id]`	GET, PATCH	`expenses.view`, `expenses.create`, `expenses.approve`	`src/app/api/vouchers/[id]/route.ts`

Server Action Files (11)

File	Exported Server Actions	Detected Permission Check(s)
`src/app/partners/capital/page.tsx`	-	`partners.capital.deposit`, `partners.manage`, `partners.view`
`src/app/partners/page.tsx`	-	`partners.manage`, `partners.view`
`src/app/projects/new/page.tsx`	-	`projects.view`, `projects.create`
`src/app/projects/plots/page.tsx`	-	`projects.view`, `plots.manage`, `projects.create`, `projects.edit`
`src/app/projects/[id]/page.tsx`	-	`projects.view`, `projects.edit`, `plots.manage`
`src/app/reports/project-pnl/page.tsx`	-	`profit.view`, `reports.view`, `profit.distribute`, `reports.export`
`src/app/sales/[id]/page.tsx`	-	`sales.view`
`src/app/settings/billing/page.tsx`	-	`billing.manage`
`src/app/treasury/accounts/page.tsx`	-	`treasury.bank.manage`, `treasury.view`
`src/app/treasury/approvals/page.tsx`	-	`treasury.view`, `treasury.contra.approve`
`src/app/treasury/page.tsx`	-	`treasury.view`, `treasury.contra.approve`, `treasury.contra.create`

App Pages (37)

Route	Type	Detected Server Permission Check(s)	File
`/audit`	Page (RSC)	`audit.view`	`src/app/audit/page.tsx`
`/change-password`	Page (RSC)	Client/Session gated	`src/app/change-password/page.tsx`
`/dashboard`	Page (RSC)	`dashboard.view`, `dashboard.customize`, `dashboard.layout.manage`, `dashboard.export`	`src/app/dashboard/page.tsx`
`/expenses/approvals`	Page (RSC)	`expenses.view`, `expenses.approve`	`src/app/expenses/approvals/page.tsx`
`/expenses/breakdown`	Page (RSC)	`expenses.view`	`src/app/expenses/breakdown/page.tsx`
`/expenses/dashboard`	Page (RSC)	`expenses.view`	`src/app/expenses/dashboard/page.tsx`
`/expenses`	Page (RSC)	`expenses.view`, `expenses.approve`, `expenses.create`	`src/app/expenses/page.tsx`
`/forgot-password`	Page (RSC)	Client/Session gated	`src/app/forgot-password/page.tsx`
`/invite/[token]`	Page (RSC)	Client/Session gated	`src/app/invite/[token]/page.tsx`
`/login`	Page (RSC)	Client/Session gated	`src/app/login/page.tsx`
`/`	Page (RSC)	Client/Session gated	`src/app/page.tsx`
`/partners/capital`	Page (RSC)	`partners.capital.deposit`, `partners.manage`, `partners.view`	`src/app/partners/capital/page.tsx`
`/partners/dashboard`	Page (RSC)	`partners.view`, `partners.manage`	`src/app/partners/dashboard/page.tsx`
`/partners`	Page (RSC)	`partners.manage`, `partners.view`	`src/app/partners/page.tsx`
`/projects/new`	Page (RSC)	`projects.view`, `projects.create`	`src/app/projects/new/page.tsx`
`/projects`	Page (RSC)	`projects.view`, `projects.create`	`src/app/projects/page.tsx`
`/projects/plots`	Page (RSC)	`projects.view`, `plots.manage`, `projects.create`, `projects.edit`	`src/app/projects/plots/page.tsx`
`/projects/[id]`	Page (RSC)	`projects.view`, `projects.edit`, `plots.manage`	`src/app/projects/[id]/page.tsx`
`/reports`	Page (RSC)	Client/Session gated	`src/app/reports/page.tsx`
`/reports/project-pnl`	Page (RSC)	`profit.view`, `reports.view`, `profit.distribute`, `reports.export`	`src/app/reports/project-pnl/page.tsx`

Route	Type	Detected Server Permission Check(s)	File
`/reports/[key]`	Page (RSC)	`reports.export`	`src/app/reports/[key]/page.tsx`
`/reset-password/[token]`	Page (RSC)	Client/Session gated	`src/app/reset-password/[token]/page.tsx`
`/sales/bookings`	Page (RSC)	`sales.view`, `sales.create`, `sales.collect`	`src/app/sales/bookings/page.tsx`
`/sales/dues`	Page (RSC)	`sales.view`, `sales.collect`	`src/app/sales/dues/page.tsx`
`/sales`	Page (RSC)	`sales.view`, `sales.create`, `sales.collect`	`src/app/sales/page.tsx`
`/sales/[id]`	Page (RSC)	`sales.view`	`src/app/sales/[id]/page.tsx`
`/settings/billing`	Page (RSC)	`billing.manage`	`src/app/settings/billing/page.tsx`
`/settings/fiscal-years`	Page (RSC)	`dashboard.settings.manage`, `roles.manage`, `users.manage`	`src/app/settings/fiscal-years/page.tsx`
`/settings/roles`	Page (RSC)	`roles.manage`	`src/app/settings/roles/page.tsx`
`/settings/users`	Page (RSC)	`users.manage`	`src/app/settings/users/page.tsx`
`/signup`	Page (RSC)	Client/Session gated	`src/app/signup/page.tsx`
`/treasury/accounts`	Page (RSC)	`treasury.bank.manage`, `treasury.view`	`src/app/treasury/accounts/page.tsx`
`/treasury/approvals`	Page (RSC)	`treasury.view`, `treasury.contra.approve`	`src/app/treasury/approvals/page.tsx`
`/treasury/dashboard`	Page (RSC)	`treasury.view`	`src/app/treasury/dashboard/page.tsx`
`/treasury`	Page (RSC)	`treasury.view`, `treasury.contra.approve`, `treasury.contra.create`	`src/app/treasury/page.tsx`
`/treasury/transactions`	Page (RSC)	`treasury.view`	`src/app/treasury/transactions/page.tsx`
`/vouchers`	Page (RSC)	Client/Session gated	`src/app/vouchers/page.tsx`

10. Reports & Dashboard

Purpose: Complete catalog of all 26 built-in reports, 6 dashboard tabs, and undefined dashboard widgets, plus formatting conventions.

Intended Reader: Product Owner, Accountant, Frontend/Backend Developers.

Last Verified: 2026-11-12

Number, Currency & Fiscal Year Conventions

- **Currency:** Bangladeshi Taka (৳), formatted with `formatBDT()` in `src/lib/format.ts` using `en-IN` locale for **lakh** (1,00,000) and **crore** (1,00,00,000) grouping.
- **Bengali Numerals:** `toBnNum()` converts digits `0-9` to `০-৯` when `lang === "bn"`.
- **Fiscal Year:** Standard Bangladesh Fiscal Year runs **July 1 – June 30**, dynamically configurable per tenant via `/settings/fiscal-years` (`dashboard_settings` key `fiscal_years`). Reports with `preset=this_fy` read the active tenant FY in both the UI page (`src/app/reports/[key]/page.tsx`) and export API (`src/app/api/reports/[key]/route.ts`).
- **Exports:** Every report supports **PDF** (via Puppeteer HTML-to-PDF) and **Excel** (`.xls` HTML table) at `/api/reports/[key]?format=pdf|excel` .

Report Catalog (26 Reports in `src/lib/reports/registry.ts`)

Key	Title (EN)	Title (BN)	Category	Required Permission	Filters
`balance-sheet`	Balance Sheet	উদ্বৃত্তপত্র / ব্যালেন্স শিট	financial	`reports.financial.view`	dateRange, project
`cash-book`	Cash Book	নগদ খাতা	financial	`reports.financial.view`	dateRange
`bank-book`	Bank Book	ব্যাংক খাতা	financial	`reports.financial.view`	dateRange, bankAccount
`daily-summary`	Daily Collection & Payment Summary	দৈনিক আদায় ও পরিশোধ সারাংশ	financial	`reports.financial.view`	dateRange
`receipts-payments`	Receipts & Payments Statement	আদায় ও পরিশোধ বিবরণী	financial	`reports.financial.view`	dateRange
`contra-register`	Contra / Fund Transfer Register	কন্ট্রা / তহবিল স্থানান্তর নিবন্ধন	financial	`reports.financial.view`	dateRange
`partner-capital`	Partner Capital Statement	অংশীদার মূলধন বিবরণী	partners	`reports.partners.view`	dateRange, partner
`partners-summary`	All Partners Summary	সকল অংশীদার সারাংশ	partners	`reports.partners.view`	None
`partner-contribution`	Partner Contribution by Project	প্রকল্পভিত্তিক অংশীদার অবদান	partners	`reports.partners.view`	project

Key	Title (EN)	Title (BN)	Category	Required Permission	Filters
`project-cost`	Project Cost Statement	প্রকল্প খরচ বিবরণী	projects	`reports.projects.view`	project
`project-pnl`	Project Profit & Loss	প্রকল্প লাভ-ক্ষতি	projects	`reports.projects.view`	project
`projects-comparison`	All Projects Comparison	সকল প্রকল্প তুলনা	projects	`reports.projects.view`	None
`expense-register`	Expense Register	ব্যয় নিবন্ধন	projects	`reports.projects.view`	dateRange, project, status
`pending-approval`	Pending Approval Report	অনুমোদনের অপেক্ষায় প্রতিবেদন	projects	`reports.projects.view`	project
`land-documents`	Land Document Register	জমি দলিল নিবন্ধন	projects	`reports.projects.view`	None
`sales-register`	Sales Register	বিক্রয় নিবন্ধন	sales	`reports.sales.view`	dateRange, project, status
`plot-inventory`	Plot Inventory / Stock Report	প্লট তালিকা / স্টক প্রতিবেদন	sales	`reports.sales.view`	project
`buyer-statement`	Buyer Statement	ক্রেতা বিবরণী	sales	`reports.sales.view`	sale
`receivables-aging`	Receivables Aging	বকেয়া বয়সভিত্তিক প্রতিবেদন	sales	`reports.sales.view`	dateRange, project
`installment-report`	Installment Due & Overdue Report	কিস্তি বকেয়া ও মেয়াদোত্তীর্ণ প্রতিবেদন	sales	`reports.sales.view`	dateRange
`collection-report`	Collection Report	আদায় প্রতিবেদন	sales	`reports.sales.view`	dateRange, project
`profit-allocation`	Profit Allocation Sheet	লাভ বণ্টন চার্ট	profit	`reports.financial.view`	project
`profit-history`	Profit Distribution History	লাভ বণ্টন ইতিহাস	profit	`reports.financial.view`	dateRange, project
`audit-trail`	Audit Trail Report	অডিট ট্রেইল প্রতিবেদন	audit	`reports.audit.view`	dateRange, user, table, action
`user-activity`	User Activity Report	ব্যবহারকারী কার্যকলাপ প্রতিবেদন	audit	`reports.audit.view`	dateRange, user
`reconciliation`	Reconciliation Check	মিলান পরীক্ষা	audit	`reports.audit.view`	None

Dashboard Tabs (6 Tabs)

Tab Key	Title (EN)	Title (BN)	Required Permission
`overview`	Executive Overview	এক্সিকিউটিভ ওভারভিউ	`dashboard.tab.overview`

Tab Key	Title (EN)	Title (BN)	Required Permission
`financial`	Treasury & Financials	ট্রেজারি ও আর্থিক অবস্থা	`dashboard.tab.financial`
`projects`	Projects & Land	প্রকল্প ও জমি	`dashboard.tab.projects`
`sales`	Sales & Collections	বিক্রয় ও আদায়	`dashboard.tab.sales`
`partners`	Partners & Capital	অংশীদার ও মূলধন	`dashboard.tab.partners`
`operations`	Operations & Approvals	অপারেশনস ও অনুমোদন	`dashboard.tab.operations`

Dashboard Widget Registry (32 Widgets in `src/lib/dashboard/widgets.ts`)

Widget Key	Title (EN)	Tab	Type	Required Permission
`cash_balance`	Cash in Hand	overview	kpi	`dashboard.widget.cash_balance`
`bank_balance`	Total Bank Balance	overview	kpi	`dashboard.widget.bank_balance`
`total_liquidity`	Total Liquidity	overview	kpi	`dashboard.widget.total_liquidity`
`working_capital`	Available Working Capital	financial	kpi	`dashboard.widget.working_capital`
`partner_equity`	Partner Equity Injected	partners	kpi	`dashboard.widget.partner_equity`
`capital_deployed`	Capital Deployed in Land	projects	kpi	`dashboard.widget.capital_deployed`
`unsold_inventory`	Unsold Inventory Value	projects	kpi	`dashboard.widget.unsold_inventory`
`total_receivables`	Total Receivables	sales	kpi	`dashboard.widget.total_receivables`
`overdue_receivables`	Overdue Receivables	sales	kpi	`dashboard.widget.overdue_receivables`
`sales_month`	Sales This Month	overview	kpi	`dashboard.widget.sales_month`
`collections_month`	Collections This Month	sales	kpi	`dashboard.widget.collections_month`
`expenses_month`	Expenses This Month	operations	kpi	`dashboard.widget.expenses_month`
`pending_approvals`	Pending Approvals	operations	kpi	`dashboard.widget.pending_approvals`
`net_profit`	Net Profit & Realized Cash	overview	kpi	`dashboard.widget.net_profit`
`portfolio_roi`	Portfolio ROI %	projects	kpi	`dashboard.widget.portfolio_roi`
`cash_flow_trend`	12-Month Cash Flow Trend	financial	chart	`dashboard.widget.cash_flow_trend`
`project_cost_vs_sales`	Project Cost vs Sales vs Collected	projects	chart	`dashboard.widget.project_cost_vs_sales`
`project_cost_share`	Project-wise Cost Share	overview	chart	`dashboard.widget.project_cost_share`
`expense_breakdown`	Expense Breakdown by Category	operations	chart	`dashboard.widget.expense_breakdown`

Widget Key	Title (EN)	Tab	Type	Required Permission
`receivables_aging`	Receivables Aging	sales	chart	`dashboard.widget.receivables_aging`
`plot_inventory_status`	Plot Inventory Status	projects	chart	`dashboard.widget.plot_inventory_status`
`partner_equity_dist`	Partner Equity Distribution	partners	chart	`dashboard.widget.partner_equity_dist`
`profit_by_project`	Profit by Project	financial	chart	`dashboard.widget.profit_by_project`
`collection_trend`	Collection Trend vs Scheduled Dues	sales	chart	`dashboard.widget.collection_trend`
`cash_flow_forecast`	90-Day Cash Flow Forecast	financial	chart	`dashboard.widget.cash_flow_forecast`
`recent_transactions`	Recent Treasury Transactions	financial	table	`dashboard.widget.recent_transactions`
`overdue_installments`	Overdue Installments List	sales	list	`dashboard.widget.overdue_installments`
`project_summary`	Projects Summary Table	projects	table	`dashboard.widget.project_summary`
`partner_summary`	Partners Capital & Balance	partners	table	`dashboard.widget.partner_summary`
`pending_vouchers`	Pending Expense Vouchers	operations	list	`dashboard.widget.pending_vouchers`
`quick_actions`	Quick Actions	overview	quick_actions	`dashboard.widget.quick_actions`
`active_alerts`	Smart Insights & Alerts	overview	alerts	`dashboard.widget.active_alerts`

---

11. Background Jobs & Worker Architecture

Purpose: Documents BullMQ queues, Redis requirements, tenant provisioning worker, and scheduled/missing jobs.

Intended Reader: Backend Developer, DevOps / Maintainer.

Last Verified: 2026-11-12

Queue & Worker Overview (`src/worker.ts`)

LandInvest uses **BullMQ** (`bullmq`) backed by **Redis 6.2+** (`ioredis`). Start the worker with:

```
npm run worker
```

On startup, `src/worker.ts` verifies:

1. MySQL is reachable on `localhost:3306` .
2. Redis responds `PONG` and is version **6.2+** (required for BullMQ streams / `XADD`).

Active Queues

`tenant-provisioning`

- **Trigger:** Enqueued by `POST /api/signup` when a new organization signs up.
- **Job Payload:** `{ tenantId: string, dbName: string }`
- **Steps Executed:**

1. `CREATE DATABASE IF NOT EXISTS \` ${dbName}\` CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci`

2. Runs `npx prisma db push --schema=prisma/tenant/schema.prisma --skip-generate` with `DATABASE_URL_TENANT` pointing to the new database.

3. Calls `seedDefaultCategories(dbName)` (creates `default_categories` table and inserts 6 default categories idempotently).

4. Calls `seedDefaultRoles(tPrisma, owner.id)` to create the 4 default roles (`Owner` , `Managing Partner` , `Accountant` , `Site Manager`) and assigns `Owner` to the signup `global_users` record.

- **Failure Handling & Monitoring:** Worker logs `[worker] provisioning ...` to stdout/stderr; failed jobs remain in BullMQ's failed set in Redis and can be inspected via BullMQ/Redis CLI (`LRANGE` / `XINFO`).

Unimplemented / Planned Jobs (`UNVERIFIED:`)

- **Plot Hold Expiry:** `UNVERIFIED:` No scheduled BullMQ repeatable job currently expires plot holds automatically.
- **Automated Daily Snapshots / Backups:** `UNVERIFIED:` Not present in `src/worker.ts` .

`;

writeDoc("11-background-jobs.md", ...)`

---

12. Configuration & Environment Variables

Purpose: Lists all environment variable names, database-backed settings tables, and feature flags.

Intended Reader: DevOps, New Developer, Technical Lead.

Last Verified: 2026-11-12

Environment Variables (Names Only — Never Commit Secrets)

Variable Name	Purpose	Required	Example Placeholder
`DATABASE_URL_CONTROL`	MySQL connection string for `control_plane`	Yes	`mysql://user:pass@localhost:3306/control_plane`
`DATABASE_URL_TENANT`	MySQL connection string for tenant Prisma CLI / default tenant	Yes	`mysql://user:pass@localhost:3306/tenant_demo`
`NEXTAUTH_SECRET`	Secret key for signing Auth.js JWTs	Yes	`<random-32-byte-hex-secret>`
`NEXTAUTH_URL`	Canonical base URL of the Next.js app	Yes	`http://localhost:3000`
`BASE_URL`	Used by test/acceptance scripts	For tests	`http://localhost:3000`
`REDIS_URL`	Redis connection URL for BullMQ & rate limiting	Yes (worker)	`redis://localhost:6379`
`AUTH_RATE_LIMIT_MAX`	Maximum login attempts per window	Optional	`20`
`NODE_ENV`	Runtime mode (`development` / `production`)	Standard	`development`

(Prisma-internal `PRISMA_` and `DOTENV_` variables referenced by generated clients do not need manual configuration.)

Database Settings Tables (`dashboard_settings`)

Stored in each tenant database (`dashboard_settings` : `id` , `setting_key` , `setting_value` JSON, `updated_by` , `updated_at`):

`setting_key`	Managed By	Purpose & Default
`fiscal_years`	`src/lib/fiscalYear.ts` , `/settings/fiscal-years`	Array of `FiscalYearEntry` (`id` , `label` , `labelBn` , `startDate` , `endDate` , `isCurrent` , `status`). Defaults to current & previous Bangladesh Jul–Jun FYs if unset.
`alert_thresholds` (or dashboard insight keys)	`src/lib/dashboard/insights.ts`	Thresholds for cash alerts, overdue installments, and executive notifications.

Feature Flags

- No external feature-flag service is used. Feature visibility is controlled by **RBAC permissions** (`src/lib/permissions.ts`) and tenant status (`control_plane.tenants.status` in `["active", "trial"]`).

`;

writeDoc("12-configuration-and-environment.md", ...)`

---

13. Setup and Run Guide (Clean Machine)

Purpose: Step-by-step instructions to run LandInvest locally from a fresh checkout using native Windows/Linux services (no Docker required).

Intended Reader: New Developer, QA Engineer.

Last Verified: 2026-11-12

Prerequisites

1. **Node.js 20+** (verified on `v24.19.0`) and **npm** (`11+`).
2. **MySQL 8.4** running natively on port `3306` (e.g., Windows service `MySQL84` or `mysqld.exe --datadir=... --port=3306`).
3. **Redis 6.2+** running natively on port `6379` (e.g., `redis-windows` 6.2.22 or Memurai on Windows; `redis-server` on Linux).

Step-by-Step Setup

1. **Clone & Install Dependencies**

```bash
git clone <repository-url> LandInvest
cd LandInvest
npm install
```

2. **Configure `.env`** (create `.env` from `.env.example` or set required variables):

```env
DATABASE_URL_CONTROL="mysql://root:@localhost:3306/control_plane"
DATABASE_URL_TENANT="mysql://root:@localhost:3306/tenant_demo"
NEXTAUTH_SECRET="<replace-with-random-secret>"
NEXTAUTH_URL="http://localhost:3000"
REDIS_URL="redis://localhost:6379"
```

3. **Create Databases & Push Schemas + Seed Demo Data**

```bash
npm run setup
```

(Or push Prisma schemas manually: `npx prisma db push --schema=prisma/control/schema.prisma` and `npx prisma db push --schema=prisma/tenant/schema.prisma`, then run `npm run seed:roles`, `node scripts/seed_realistic_data.mjs`, and `npm run seed:fy`.)

4. **Start the Background Worker & Next.js Dev Server**

```bash
```

Terminal 1 (Tenant provisioning worker)

npm run worker

Terminal 2 (Web application)

npm run dev

```

*(Note: `npm run dev` runs `next dev --webpack` on `http://localhost:3000`.)*

5. **Log In with Demo Accounts** (seeded in `tenant_demo` / `control_plane`):

- **Owner:** `owner@demo.com` (password set by `setup.mjs` — e.g., check `scripts/setup.mjs` for local demo password)

- **Managing Partner:** `admin@demo.com`

- **Site Manager:** `site@demo.com`

- **Accountant:** `acct@demo.com`

---

## Troubleshooting (Real Issues Observed in This Repo)

| Symptom | Root Cause | Resolution |
|---|---|---|
| `Redis X.X is too old for BullMQ` | Older Redis 3.x/5.x on Windows lacks `XADD` streams. | Use Redis 6.2+ ( `redis-windows` 6.2.22 or Memurai). |
| `MySQL is not reachable on port 3306` | `mysqld` service stopped. | Start `MySQL84` service or run `mysqld --port=3306`. |
| `EPERM: operation not permitted, mkdir` on Windows | Transient Windows filesystem lock during `npm` / `npx`. | Re-run the command once. |
| `Table 'tenant_demo.users' doesn't exist` | Login accounts live in `control_plane.global_users`, not in tenant DBs. | Query `control_plane.global_users` for auth records. |

`;

writeDoc("13-setup-and-run-guide.md", ...)`

---
```

14. Deployment & Operations

Purpose: Production build, process management, backups, multi-tenant schema migrations, and release checklist.

Intended Reader: DevOps Engineer, Technical Lead, Maintainer.

Last Verified: 2026-11-12

Production Build & Required Services

1. **Build:** `npm run build` (`next build`) followed by `npm run start` (`next start`).
2. **Required Services:**

- Next.js web process (`npm run start`)
- BullMQ worker process (`npm run worker`)
- MySQL 8.4+
- Redis 6.2+
- Chromium (for Puppeteer PDF exports)

> **Status:** `NOT IMPLEMENTED` — No production `Dockerfile`, `nginx.conf`, or `PM2` (`ecosystem.config.js`) manifest exists in the repository. **Recommendation:** Run Next.js and `npm run worker` under `pm2` or `systemd` behind Nginx/Caddy with TLS (`HTTPS`).

Backup & Restore Strategy (`UNVERIFIED:` — No automated script in repo)

- **Control Plane Backup:**

```bash
mysqldump -u <user> -p control_plane > control_plane_$(date +%F).sql
```

- **Single Tenant Backup:**

```bash
mysqldump -u <user> -p tenant_<slug> > tenant_<slug>_$(date +%F).sql
```

- **Restore One Tenant:**

```bash
mysql -u <user> -p tenant_<slug> < tenant_<slug>_2026-11-12.sql
```

Rolling Out Schema Migrations Across All Tenants

Because each tenant has its own database, any change to `prisma/tenant/schema.prisma` must be applied to **every** tenant in `control_plane.tenants`:

```
# Example rollout loop (run after backing up all databases)
# Query SELECT db_name FROM control_plane.tenants WHERE status IN ('active', 'trial')
```

```
# For each db_name:
DATABASE_URL_TENANT="mysql://<user>:<pass>@localhost:3306/<db_name>" npx prisma db push --schema=prisma/ten
```

(Also see `scripts/migrate_rbac.mjs` and `scripts/migrate_dashboard_schema.mjs` for existing multi-tenant rollout script patterns in this repo.)

Pre-Release Checklist

- [ ] `npm run typecheck` passes with zero errors.
- [ ] `npm run lint` passes.
- [ ] `npm run test:fy`, `npm run test:all`, and `npm run acceptance` pass.
- [ ] Database backups taken for `control_plane` and all active `tenant_*` DBs.
- [ ] Schema changes pushed to all active tenant DBs.
- [ ] `npm run docs:generate` re-run if tables, permissions, routes, or reports changed.

`;

writeDoc("14-deployment-and-operations.md", ...)`

---

15. Security

Purpose: Documents tenant isolation, authentication/session security, RBAC, PII handling, audit trail coverage, and remaining security risks.

Intended Reader: Security Auditor, Technical Lead, Business Owner.

Last Verified: 2026-11-12

---

1. Tenant Isolation

- **Physical Isolation:** Every customer organization has its own MySQL schema (`tenant_<slug>`).
- **Session-Derived Resolution:** `getTenantClientFromSession()` (`src/lib/tenant.ts`) reads `tenantId` exclusively from the signed Auth.js JWT (`getServerSession`), never from a request query/body parameter.
- **Live Suspension Check:** `src/auth.ts` re-queries `control_plane.global_users.isActive` and `tenants.status` on every session check (fail-closed if DB unreachable or user/tenant deactivated).

2. Role-Based Access Control (RBAC)

- Every API route and sensitive RSC page calls `can(session, "<permission.key>")` (`src/lib/rbac.ts`).
- `Owner` (`isOwner: true`) bypasses permission checks; all other roles are checked against `tenant.user_roles` -> `role_permissions` .
- Verified across 4 roles (`owner@demo.com` , `admin@demo.com` , `site@demo.com` , `acct@demo.com`) by `scripts/acceptance.mjs` (Phase C4).

3. Audit Trail Coverage

- `writeAudit()` (`src/lib/tenantRules.ts`) logs `table_name` , `record_id` , `action` , `changed_by` , `old_values` , and `new_values` into `audit_log` .
- Verified by `scripts/acceptance.mjs` (Phase C3): 100% of `project_expenses` , `treasury_ledger` , `land_sales` , `customer_payments` , and `profit_distributions` rows have matching `audit_log` entries with non-null `changed_by` .

4. Personal Data (PII) & File Uploads

- **PII Stored:** Buyer `buyer_name` , `buyer_phone` , `buyer_nid` (`land_sales`), Partner `phone` , `nid_number` , `bank_account_number` (`partners`), and User `email` (`global_users`).
- **File Uploads:** Expense vouchers uploaded via `POST /api/expenses` are saved via `src/lib/storage.ts` with UUID prefixes (`vouchers/${randomUUID()}-${file.name}`).

5. Candid List of Remaining Risks

1. **Hardcoded Local MySQL Credentials in `tenant.ts` / `worker.ts` :** `src/lib/tenant.ts` currently constructs `mysql://${tenant.dbUser}:@localhost:3306/${tenant.dbName}` (empty password) and `src/worker.ts` uses `{ host: "localhost", user: "root", password: "" }` . Must be parameterized via encrypted credentials / env vars for production.

2. **File Upload Validation:** `POST /api/expenses` saves uploaded voucher files without strict MIME-type/magic-byte or file-size allowlist checks.

3. **17 `npm audit` Vulnerabilities:** `npm audit` reports 1 critical, 9 high, and 7 moderate advisories across the dependency tree (`03-tech-stack.md`).

4. **Stubbed Billing & Password Reset Delivery:** Billing payment (`POST /api/billing/pay`) has no cryptographic webhook verification because no live gateway is wired yet.

`;

writeDoc("15-security.md", ...)`

---

16. Testing

Purpose: Documents all automated test suites, how to run them, what they cover, and remaining coverage gaps.

Intended Reader: QA Engineer, New Developer, Technical Lead.

Last Verified: 2026-11-12

Test Suites & Commands

Script	Command	What It Tests	Latest Observed Result
TypeScript Check	`npm run typecheck`	Full strict TypeScript compilation (`tsc --noEmit`).	Pass (exit 0)
Acceptance Suite	`npm run acceptance`	92 checks across C1 (MySQL/Redis/Next), C2 (Expenses, Sales, FIFO installments, Contra, PnL, CSV/PDF), C3 (Audit trail completeness & Paid-by-Partner isolation), C4 (RBAC across 4 roles).	**92/92 passed**
Fiscal Year TDD	`npm run test:fy`	5 E2E tests (`node:test`) covering unauth 401, RBAC, date/duplicate/delete validation, MySQL + audit persistence, and `preset=this_fy` integration across Reports UI + Export API.	**5/5 passed**
Full Project E2E	`npm run test:all`	8 E2E suites (`node:test`) covering Login, all 29+ pages, core read APIs, Contra, Expenses/Vouchers, Multi-plot & Full-payment Sales + Overpayment guards, Reports catalog, and all 26 Reports + PnL exports.	**8/8 passed**
i18n Key Check	`npm run test:i18n`	Verifies English/Bengali translation key parity (`scripts/test_translation_keys.mjs`).	Available
RBAC Smoke	`npm run rbac:smoke`	Quick RBAC smoke checks (`scripts/rbac_smoke.mjs`).	Available

Coverage Gaps

1. **Headless Browser UI Tests:** Current E2E suites test HTTP/HTML/API/DB layers via `fetch` + `mysql2` ; there is no Playwright/Cypress DOM click-through suite for client-only modal interactions.
2. **Plot Hold Expiry & Land Acquisition Wizard:** Multi-step wizard edge cases (e.g., partner share != 100% UI validation) rely primarily on manual/UI checks rather than automated unit tests.
3. **Production Load / Concurrency Testing:** No stress test for concurrent multi-tenant `PrismaClient` connection pooling.

`;

writeDoc("16-testing.md", ...)`

---

17. Developer Guide

Purpose: Coding conventions, core helper patterns, and step-by-step recipes for extending LandInvest safely.

Intended Reader: New Developer, Maintainer.

Last Verified: 2026-11-12

Core Coding Patterns

1. **Tenant DB Access:** Always call `const prisma: any = await getTenantClientFromSession();` inside authenticated API routes and Server Components — never accept a tenant DB name or ID from client input.
2. **Permission Checks:** Always check `if (!(await can(session, "<permission.key>")))` on both the UI page (rendering `Forbidden: <key>`) and the mutating API/Server Action (`403`).
3. **Atomic Financial Writes + Audit:** Wrap all multi-table financial writes in `prisma.$transaction(async (tx) => { ... })` and call `await writeAudit(tx, tableName, recordId, action, userId, oldVal, newVal)`.
4. **Decimal Math & Formatting:** Use `toNum()` when reading Prisma `Decimal` fields and `formatBDT()` / `formatCompactBDT()` from `src/lib/format.ts` for display.

Step-by-Step Extension Recipes

Task	Files to Touch	Notes
Add a New Permission	`src/lib/permissions.ts`, `src/lib/seedRoles.ts`	Add key to `BasePermissionKey` & `PERMISSIONS`; assign to appropriate default roles; run `npm run seed:roles`.
Add a New Report	`src/lib/reports/<key>.ts`, `src/lib/reports/index.ts`, `src/lib/reports/registry.ts`	Export `query<Name>(prisma, filters)` returning `ReportData`, register in `REPORT_QUERIES` and `REPORT_REGISTRY`.
Add a Dashboard Widget	`src/lib/dashboard/widgets.ts`, `src/lib/permissions.ts`, `src/lib/dashboard/`	Register widget key, tab, type, and query function; permission `dashboard.widget.<key>` is auto-derived.
Add a Table or Column	`prisma/tenant/schema.prisma`, `src/worker.ts`	Update Prisma schema, run `npx prisma generate --schema=prisma/tenant/schema.prisma`, push to all existing tenant DBs (`14-deployment-and-operations.md`).
Add a Language Label (EN/BN)	`src/lib/i18n.tsx`	Add `{ en: "...", bn: "..." }` to `DICT` and verify with `npm run test:i18n`.
Add a Fiscal Year / Setting	`src/lib/fiscalYear.ts` (or `dashboard_settings`)	Store under a unique `settingKey` in `dashboard_settings` with `writeAudit()`.

`;

writeDoc("17-developer-guide.md", ...)`

---

18. Architecture & Design Decision Log

Purpose: Records key architectural and product decisions, alternatives considered, and trade-offs.

Intended Reader: Technical Lead, New Developer, Business Owner.

Last Verified: 2026-11-12

#	Decision	Choice Made	Alternatives Considered	Reason & Evidence	Trade-Off
D-01	Multi-tenancy isolation	**Database-per-tenant** (`control_plane` + `tenant_<slug>`)	Shared database with `tenant_id` row-level filtering	`Inferred reason:` Hard physical isolation between competing land syndicates and per-tenant backup/restore.	Schema changes must roll out to every tenant DB (`src/lib/tenant.ts`).
D-02	Primary Database	**MySQL 8.4**	PostgreSQL	Configured across `prisma/control/schema.prisma`, `prisma/tenant/schema.prisma`, and `mysql2` scripts.	Cross-DB foreign keys are enforced in app code rather than SQL constraints.
D-03	ORM Setup	**Prisma with dual schemas**	Single schema / raw SQL only	Separate `prisma/control` and `prisma/tenant` schemas generate isolated clients (`src/generated/*`).	Two `prisma generate` targets to maintain.
D-04	Full-stack Framework	**Next.js 16 App Router + Server Actions + API Routes**	Separate SPA + Express backend	Unified TypeScript codebase with server-side RBAC gating (`src/app/`).	Dev server uses `next dev --webpack` on Windows.
D-05	Authentication	**Auth.js v4 Credentials + JWT**	OAuth-only / Database sessions	Simple email/password login for syndicate staff with live `isActive` DB re-check (`src/auth.ts`).	Requires `NEXTAUTH_SECRET` and manual password reset token handling.
D-06	Primary Keys	**UUID v4 (`CHAR(36)`)**	Auto-increment integers	Application-generated `randomUUID()` allows pre-generating IDs for audit logs inside transactions.	Slightly larger index size than `BIGINT`.
D-07	Land Area Base Unit	**Shotok (`DECIMAL`)**	Sqft / Katha as base	Standard Bangladeshi unit across rural/peri-urban land (`project_plots.plot_size_shotok`, `src/lib/area.ts`).	UI converts to Katha/Bigha on display.
D-08	Treasury Balances	**Ledger-derived (`treasury_ledger`)**	Mutable balance columns only	Immutable `CREDIT` / `DEBIT` ledger guarantees full auditability (`src/lib/treasury.ts`, `src/lib/collections.ts`).	`bank_accounts.current_balance` must stay synced in `$transaction`.
D-09	Dashboard RBAC	**Per-tab & per-widget permissions**	Single all-or-nothing dashboard flag	Allows Owners to grant Accountants financial widgets without exposing executive profit charts (`src/lib/permissions.ts`).	79 permission keys to manage.

#	Decision	Choice Made	Alternatives Considered	Reason & Evidence	Trade-Off
D-10	Acquisition Costs	**Reuse `project_expenses`**	Separate acquisition cost table	Bayna, registration, and development costs flow through one voucher approval & ledger pipeline (`src/app/api/expenses/route.ts`).	Relies on `expense_category` to distinguish land vs. development costs.
D-11	Local Infrastructure	**Native MySQL + Redis on Windows**	Docker Compose	`Inferred reason:` Explicit native service checks in `src/worker.ts` (`net start MySQL84` , `redis-windows 6.2+`).	Contributors must install MySQL & Redis locally.
D-12	SaaS Billing	**Stubbed billing module**	Live Stripe / SSLCommerz / bKash	`Inferred reason:` Data model (`billing_invoices` , `subscription_plans`) is ready while payment gateway integration is deferred.	Cannot collect real online payments yet.

`;

writeDoc("18-decision-log.md", ...)`

---

19. Known Issues, Tech Debt & Roadmap

Purpose: Candid inventory of incomplete/stubbed features, technical debt, dependency advisories, and prioritized next steps.

Intended Reader: Technical Lead, Business Owner, Maintainer.

Last Verified: 2026-11-12

Severity-Ranked Issues & Tech Debt

Severity	Item	Where	Details & Recommended Fix
High	**Hardcoded DB Credentials in `tenant.ts` & `worker.ts`**	`src/lib/tenant.ts`, `src/worker.ts`	Uses `root` with empty password on `localhost`. Parameterize host/user/password from encrypted tenant config or env vars before production.
High	**Stubbed Billing Payment Gateway**	`src/app/api/billing/pay/route.ts`, `/settings/billing`	Marks invoices paid without live gateway verification. Integrate SSLCommerz / bKash / Stripe + signed webhooks.
High	**Schema Mismatch: `default_categories`**	`src/worker.ts` vs `prisma/tenant/schema.prisma`	`default_categories` table is created via raw SQL in `src/worker.ts` and missing from `prisma/tenant/schema.prisma`. Add it to Prisma schema or remove raw DDL.
Medium	**17 `npm audit` Vulnerabilities**	`package-lock.json` (`docs/_generated/npm_audit.json`)	1 critical, 9 high, 7 moderate advisories. Review and upgrade affected packages (`npm audit fix` / targeted bumps).
Medium	**No Bounded Pool / Eviction on `tenantClients`**	`src/lib/tenant.ts`	`globalThis.tenantClients` grows unboundedly as active tenants increase. Add LRU cache with `$disconnect()` eviction and `connection_limit`.
Medium	**Upload MIME/Size Validation**	`src/app/api/expenses/route.ts`	Voucher uploads should enforce allowed MIME types (`image/*`, `application/pdf`) and max file size.
Low	**No Automated Plot Hold Expiry Worker**	`src/worker.ts`	Only `tenant-provisioning` worker is active; add scheduled BullMQ job if temporary plot holds are used.
Low	**Production Deployment Manifests Missing**	Repository root	Add `ecosystem.config.js` (PM2), Nginx reverse proxy template, and automated `mysqldump` backup script.

Suggested Next Steps (Priority Order)

1. Parameterize MySQL credentials and `PrismaClient` connection limits in `src/lib/tenant.ts` and `src/worker.ts`.
2. Add `default_categories` to `prisma/tenant/schema.prisma` so Prisma and MySQL are 100% in sync.
3. Wire live transactional email (invitations & password resets) and payment gateway (billing).

4. Add Playwright browser E2E tests for client modals and set up automated daily DB backups.

`;

writeDoc("19-known-issues-and-roadmap.md", ...)`

---

20. Glossary (English & Bengali)

Purpose: Plain-language definitions of Bangladeshi real estate terms and software/accounting terms used in LandInvest.

Intended Reader: Non-technical Owner, New Developer, Stakeholder.

Last Verified: 2026-11-12

Bangladeshi Land & Real Estate Terms

Term (English)	Bengali (বাংলা)	Meaning
Bayna	বায়না	Earnest money / advance booking deposit paid to lock a land or plot purchase.
Dag	দাগ	Plot/survey parcel number recorded on a Mouza map.
Khatian (CS, SA, RS, BS)	খতিয়ান	Official land ownership record-of-rights across historical surveys (Cadastral, State Acquisition, Revisional, Bangladesh Survey).
Mouza	মৌজা	Lowest revenue/cadastral geographical land unit (village/estate survey area).
JL (Jurisdiction List) No.	জে.এল. নং	Unique number identifying a Mouza within a Thana/Upazila.
Namjari / Mutation	নামজারি	Updating government land records to register the new owner's name after purchase.
Khajna	খাজনা	Annual land development tax paid to the Union Land Office.
Shotok (Decimal)	শতক	Standard land area unit (`1/100` of an acre = `435.6 sq ft`); base unit in LandInvest.
Katha	কাঠা	Traditional urban land unit (`1 Katha ≈ 1.65 Shotok` / `720 sq ft`).
Bigha	বিঘা	Large land unit (`1 Bigha = 20 Katha ≈ 33 Shotok`).
Saf Kabala	সাফ কবলা	Registered final sale deed transferring full legal title to the buyer.

Accounting & Technical Terms

Term	Bengali (বাংলা)	Meaning
Contra Voucher	কন্ট্রা ভাউচার	Internal transfer of funds between Cash-in-Hand and Bank (neither income nor expense).
Money Receipt	মানি রিসিট	Official acknowledgement issued to a buyer upon receiving an advance or installment.
Receivable	প্রাপ্য	Remaining agreed amount owed by plot buyers (`current_due_amount`).
ROI (Return on Investment)	বিনিয়োগের বিপরীতে আয়	Net profit divided by total project cost, expressed as a percentage.
Fiscal Year (FY)	অর্থবছর	12-month financial reporting period (Bangladesh standard: July 1 – June 30).
Tenant	টেন্যান্ট / প্রতিষ্ঠান	An isolated customer organization with its own dedicated MySQL database.

Term	Bengali (বাংলা)	Meaning
Control Plane	কন্ট্রোল প্লেন	The shared central database (`control_plane`) managing logins, tenants, and billing.
RBAC	ভূমিকা-ভিত্তিক প্রবেশাধিকার	Role-Based Access Control — assigning granular permissions to roles rather than individual pages.
Ledger	খতিয়ান / লেজার	Append-only record of `CREDIT` and `DEBIT` financial movements.
Idempotent	আইডেমপোটেন্ট	An operation that produces the same safe result even if executed multiple times (e.g., `seedRoles`).

`;

writeDoc("20-glossary.md", ...)`